

Valmo Advocate

Partner Support Platform — System Documentation

Prepared for: Analyst & Functional Leadership

Prepared by: Valmo Advocate — Engineering

Date: 21 July 2026

Version: 1.0

Classification: Internal — Meesho / Valmo

Contents

1. Executive Overview & Problem Statement
 2. System Architecture & Technology Stack
 3. The Resolution Engine (agentic loop)
 4. Trust Spine & Adversarial Verifier
 5. Knowledge Model & SOP Compiler
 6. Domain Brains & the Authoring Studio
 7. Governance Framework & Conformance Loop
 8. L3 Escalation Platform
 9. Concern Log, Audit & Continuous Problem Discovery
 10. Data Substrate & Durability
 11. Proactive Monitoring
 12. Authentication, Roles & Security
 13. Channels & Voice / Conversation Mode
 14. Deployment & Operations
 15. Evaluation & Results
- Appendix A — Recent Hardening & Feature Cycle

About this document

This document is a step-by-step technical and functional walkthrough of the Valmo Advocate partner-support platform, written for analysts and functional leaders who need to understand what has been built, how each part works, and what is production-ready versus demonstration-only.

Each section is grounded in the actual source code, not a marketing description. Where a capability is a working build it is described as such; where something is a placeholder, a stub, canned/seed data, or a design that has not yet been built, that is stated plainly under a “Status” note. This honesty is deliberate: the platform today is a runnable slice of a larger design — a genuinely working reasoning-and-trust core, with several external integrations (live database, real WhatsApp round-trip, actual money movement) intentionally stubbed pending sign-off.

Two companion documents in the repository give further context: `PRODUCT_VISION.md` (the aspirational north star) and `PRODUCTION_DELTA.md` (the honest build ledger of demo-vs-production). Where those documents and the live configuration disagree, the live configuration and this document take precedence — the active reasoning model today is OpenAI gpt-5.5 on every tier via the gateway, with Claude and Gemini configured as swappable alternatives. Appendix A logs the most recent hardening cycle.

1. Executive Overview & Problem Statement

Valmo Advocate (in-repo: the "Valmo Partner Support Platform") is a support platform for Valmo — Meesho's logistics arm — that lets delivery-partner "captains" raise any issue in natural Hinglish conversation, by voice or text, and have it resolved inside that conversation in seconds instead of filing a ticket. The docs position it explicitly as a "resolution engine, not a ticketing system," built to replace the Kapture CRM ticketing workflow that L1/L2/L3 human agents use today (the ingested SOPs are full of "escalate on kapture" steps). Its core thesis is to ground each answer in real captain data, gate any money-moving action (e.g. reversing a wrong debit) behind a trust spine plus an adversarial verifier, then act in-conversation with a full audit trail — displacing a Kapture spend the docs cite at █1.2 Cr/quarter (█4.8 Cr/year). Critically, what exists in this repo is a runnable hackathon demo — a "slice" of a larger BRD-grade design — with a genuinely working reasoning-and-trust core, but with the external systems (live DB, real WhatsApp round-trip, actual money movement, auth) deliberately stubbed or fed canned/snapshot data.

How it works

- 1 A captain raises an issue in conversation** — Free-form Hinglish (voice or text). README's hero example is 'galat debit █1240, reversal karo...' (wrong debit, reverse it). The engine is designed channel-agnostic: the demo home is a standalone React Captain Panel; a WhatsApp webhook adapter also exists but only runs locally (outbound is echoed, not sent to Meta).
- 2 An LLM-driven agentic loop understands the need and calls deterministic tools** — Per PRODUCTION_DELTA SJ (v4.0), the reactive path is an LLM tool-use loop (app/engine/conversation.py), not a hardcoded state machine: the model converses and calls tools (search_sops, get_captain_context, run_data_query, and apply_policy). 'Intelligence in the model; guarantees in the tools.'
- 3 The claim is grounded in real captain data** — A Captain Context service composes account data plus Log10 scans/shipments. In the demo these come from a canned DemoDataProvider and a real PII-stripped SQLite loss snapshot (~37k rows, backend/data/valmo.db) rather than live Metabase/Log10 — the seam to swap in the live MetabaseProvider is documented.
- 4 A disposition is located and its Executable Policy runs** — The system matches the issue to a disposition (hero case: hardstop_loss; 16 total, 15 derived from the loss dataset's reason_l1 categories) and runs that disposition's compiled policy checks in code against the grounded data — the LLM does not decide the money outcome.
- 5 The trust spine gates any money move** — Grounded generation (every claim cites a source row), a policy-as-code cap, a confidence gate (fixed 0.80 threshold in the demo), and a genuine adversarial verifier that can and does refute when evidence is weak. apply_policy is the single money path.
- 6 Resolve in-conversation, or escalate** — If checks + gate + verifier pass, the debit is reversed idempotently (in the demo this records an idempotent write — nothing real moves) and the captain gets a warm Hinglish reply citing the evidence, with no ticket. Otherwise it escalates to an L3 functional-team inbox (SLA timer, breach ladder).
- 7 Everything is logged and learned from** — Each resolution is an immutable event in an append-only Concern Log (JSON files in the demo, Postgres in production); a satisfaction prompt (█/█) feeds a CPD loop, and a cheap-rule-first proactive monitor scans a captain for risks before invoking an LLM.

Key points

- Who it serves: Valmo's last-mile delivery-partner 'captains'. The docs frame scale as '10k' (PRODUCTION_DELTA references '10k scale' / '10k concurrent' for monitoring and persistence); the

exact captain headcount is not asserted in the code, so treat ~10k as the stated scale target rather than a counted figure.

- The problem it replaces: the Kapture CRM ticketing system. README states 'Kapture is being replaced by this platform,' and the ingested SOPs repeatedly reference L1→L2→L3 escalations handled 'on kapture' — i.e. today issues become tickets that humans work.
- The core thesis, verbatim in spirit: 'resolved inside the conversation, in seconds' vs. filing a ticket — the demo script's punchline is literally 'No ticket.'
- Economics of the displacement: PRODUCTION_DELTA §J cites Kapture at ■1.2 Cr/quarter = ■4.8 Cr/year (a firmed-up figure, previously a ■4–8 Cr/yr range). This is the cost the platform is pitched to displace.
- Active model today: OpenAI gpt-5.5 on every tier (fast == deep) via a hackathon gateway — confirmed by both README and backend/config/models.yaml. The fast/deep tier split is wired but a no-op today; Claude and Gemini are config-swappable alternatives (flip the provider: key).
- Knowledge provenance is deliberately narrow: the ONLY thing taken from the existing input-bot / valmo-l1-agent / valmo-platform repos is SOP/knowledge data (several hundred chunks) — no Kapture, Metabase, scraping, or L1 code is imported. The swap seam is one file, backend/config/sources.yaml.
- Two documentation layers to read together: PRODUCT_VISION.md is the aspirational 'north star' (with a mostly-not-yet status table), while PRODUCTION_DELTA.md is the honest build ledger of demo-vs-production.

Status

DEMO vs PRODUCTION must be stated plainly for leadership: this repo is a hackathon demo, self-described as 'a runnable slice of the BRD's resolution engine.'
GENUINELY BUILT AND WORKING (per PRODUCTION_DELTA §B, and consistent with the code layout): the deterministic reasoning pipeline / agentic tool loop; Executable Policies that run real checks in code; the trust spine (grounded generation, policy-as-code cap, confidence gate, a real adversarial verifier that actually flips); the append-only Concern Log with live stats; the SOP Compiler (plain text → policy); proactive monitoring (cheap-rule-first); tiered model routing; knowledge ingestion from the three source repos; intent routing so only concrete action requests touch a money policy; and a real loss-data layer against a PII-stripped SQLite snapshot of the Metabase loss export.
STUBBED / CANNED / DESIGN-ONLY — do NOT present as live: real WhatsApp (adapter exists but cannot complete the round-trip to Meta in the sandbox — no public URL, account, or token; outbound is echoed); live Meesho DB / Metabase and Log10 (canned DemoDataProvider + connectors; the MetabaseProvider is a stub/seam); real money movement (apply_policy records an idempotent write but nothing actually moves, and it must stay behind finance sign-off); authentication/identity (none — captain is picked from a dropdown); persistence (JSON files, not Postgres/pgvector/Redis — 'fine for demo, not for 10k concurrent'); retrieval (lexical/keyword scorer, not embeddings); calibrated confidence (fixed 0.80, not recalibrated); multimodal file/image/audio ingest (not built — 'next core build'); and the governance framework (severity/recoverability/scalability), which is an explicit placeholder that returns a stub.
TWO DOC INCONSISTENCIES I VERIFIED AND SHOULD BE FLAGGED (they can mislead a reader): (1) PRODUCTION_DELTA §A still lists the demo's reasoning model as Gemini 2.5-flash/pro and §K describes a gpt-4o+gpt-5.5 gateway split, but the authoritative live config (backend/config/models.yaml) and the README both say the active provider is OpenAI running gpt-5.5 on every tier — §A is stale. (2) PRODUCT_VISION.md's status table marks 'stateful multi-turn conversation,' 'data-query path,' 'WhatsApp,' 'L3 platform,' 'KT engine,' and 'audit/CPD' as not-yet/partial, whereas PRODUCTION_DELTA §B/§J (the later v4.0 build) claims these as built; the ledger post-dates the vision doc, so for current state trust PRODUCTION_DELTA over the PRODUCT_VISION table. Minor: the ingested-knowledge chunk count is cited as 422 (README/PRODUCT_VISION/§B) and as 61 policy + 405 procedure = 466 (§J) — the exact number drifts across doc versions.

Key files

README.md · PRODUCT_VISION.md · PRODUCTION_DELTA.md · backend/config/models.yaml ·
backend/config/sources.yaml · backend/data/knowledge_sources/collated_sops.md

2. System Architecture & Technology Stack

Valmo Advocate is a single deployable web service: a Python FastAPI backend that both exposes the `/api` surface and serves a compiled React (Vite) single-page app from the same origin. The engine's two live workflows — reactive resolution (chat) and proactive monitoring — stream their step-by-step trace to the browser over Server-Sent Events so the UI can show the engine working in real time. Every large-language-model (LLM) call goes through one provider-neutral gateway and a config-driven "tiered registry" that lets the team swap between OpenAI, Claude, and Gemini by editing a YAML file (no code change). The app ships as one multi-stage Docker image deployed on Render's free tier, and because that tier has no persistent disk, all authored/operational state is mirrored to a Turso (libSQL-over-HTTPS) database so it survives restarts and redeployments.

How it works

- 1 One FastAPI app is the whole backend** — `backend/app/main.py` defines a single FastAPI application (title 'Valmo Partner Support Platform', version 'demo-1.0'). It declares every `/api` route, wires CORS wide open (`allow_origins=['*']`), and gates access with a role system: two routes are public (POST `/api/auth/login` and GET `/api/health` for Render's health check); everything else requires a valid 12-hour signed token, with 'author' vs 'approver' role gates on write/approve routes (approver implicitly satisfies author). On startup it seeds the initial admin user plus baseline knowledge (domain 'brains' and the COD Shortfall SOP).
- 2 React/Vite frontend, served two ways** — The UI is React 18 built with Vite 5, styled with Tailwind CSS, using `framer-motion` and `lucide-react` (`frontend/package.json`). In local dev (`run.sh`), Vite serves the UI on `:5190` and proxies `/api` to the backend on `:8077` (`vite.config.js`). In production the Vite build output (`frontend/dist`) is served directly by FastAPI: `main.py` mounts `/assets` and adds an SPA catch-all that returns `index.html` for unknown non-`/api` paths, so the app is single-origin (API + UI on one host), which is what makes cross-origin SSE unnecessary.
- 3 Two workflows stream over Server-Sent Events (SSE)** — POST `/api/chat` (reactive resolution, stateful multi-turn) and GET `/api/monitor/{captain_id}` (proactive monitoring) both return an `sse_starlette.EventSourceResponse`. A helper (`_sse` in `main.py`) wraps a Python generator, emitting one `{event:'trace'}` frame per pipeline step and a final `{event:'end'}` frame. Two authoring endpoints (`/api/sop/compile`, `/api/blueprint/compile`) stream the same way. The browser does NOT use the native `EventSource` (which is GET-only); `frontend/src/lib/api.js` reads the stream via `fetch + ReadableStream.getReader()`, which supports POST SSE, carries the auth token, and bounces to login on a 401.
- 4 Reactive turn = an LLM-driven bounded agentic loop** — A chat POST is auth-gated then handed to `conversation.handle_turn` (`backend/app/engine/conversation.py`). This is an LLM-run tool-use loop capped at `MAX_STEPS=6` (not a hardcoded state machine): the model converses in the captain's language and calls tools — `search_sops`, `get_captain_context`, `run_data_query`, and `apply_policy`. `apply_policy` is the SOLE money path: deterministic code runs the Executable-Policy checks, the trust gate, an adversarial verifier, and an idempotent write. Each model step and tool call is emitted as a trace event; the turn ends on a single 'reply' event with the captain-facing message, and the resolution is appended to the Concern Log.
- 5 All model calls go through one provider interface** — No pipeline code imports a vendor SDK. Every call goes through `LLMProvider` (`backend/app/llm/base.py`), and code asks the registry for a provider+model by pipeline-node name. `backend/app/llm/registry.py` reads `backend/config/models.yaml` once (cached), takes the active ``provider`` key, maps the requested node to a tier (fast/deep), and returns the concrete model id for that provider+tier. Swapping providers or re-tiering a node is a YAML edit, not a code change.

- 6 **Tiered registry: fast vs deep, currently OpenAI gpt-5.5** — `models.yaml` assigns each node a tier: `intent/classify/explain/monitor_compose = fast`; `policy_reasoning/adversarial_verify/sop_compile = deep`. The active provider is 'openai' with BOTH tiers mapped to gpt-5.5 today, so the fast/deep split is wired but collapses to one model right now. The file also lists ready-to-use tier ids for gemini (2.5-flash / 2.5-pro) and claude (haiku-4-5 / opus-4-8) as swappable alternatives. All three providers are implemented (`openai_provider.py`, `gemini_provider.py`, `claude_provider.py`); OpenAI is the one actually running.
- 7 **The active provider talks to a gateway with WAF/flake handling** — `OpenAIProvider` (`backend/app/llm/openai_provider.py`) is an OpenAI-compatible Chat Completions client built on `requests` (no SDK), defaulting to the hackathon Bifrost gateway (`gateway-buildathon.ltl.sh`), overridable via `OPENAI_BASE_URL`. It sends a full browser header set to get past an Akamai edge WAF that 403s non-browser clients, retries the gateway's intermittent 401 'virtual_key_not_found' flake up to 30x, backs off on WAF 403s and 429/5xx, omits `temperature` for gpt-5/o-series models (which reject it), and exposes a `chat()` method that translates the engine's Gemini-shaped tool-call history into OpenAI messages/tools and back so the agent loop is provider-agnostic.
- 8 **Packaged as one multi-stage Docker image** — Dockerfile stage 1 (`node:20-slim`) runs `npm ci + npm run build` to produce the React bundle. Stage 2 (`python:3.12-slim`) installs backend deps, copies the backend source, and copies the built SPA from stage 1 into `/app/frontend/dist`. The result is ONE image where FastAPI serves the SPA at `/` and the API at `/api/*`. Secrets and the large loss database are excluded via `.dockerignore` (injected via `env` / read from Turso in prod). CMD runs `uvicorn` on `$PORT` (default 8077).
- 9 **Deployed on Render's free tier via a blueprint** — `render.yaml` declares one Docker web service ('`valmo-psp`', `plan: free`, no card required) with `healthCheckPath /api/health`; Render injects `$PORT`. Secrets are set in the Render dashboard with `sync:false` so they never enter git (`OPENAI_API_KEY`, `OPENAI_BASE_URL`, `TURSO_DATABASE_URL`, `TURSO_AUTH_TOKEN`, `AUTH_SECRET`, `INITIAL_ADMIN_EMAIL/PASSWORD`). Documented free-tier caveats: the instance sleeps after ~15 min idle (slow first request) and the local `/data` dir resets on every redeploy.
- 10 **Turso provides durable state across restarts** — Because the free tier has no persistent disk, `backend/app/durable_state.py` mirrors every mutable JSON store (users, blueprints, KT queue/SOPs, governance framework, audit rubric/scores, concern log, traces, CPD) into a Turso key-value table (`psp_state`). The local file is a fast write-through cache; Turso is the durable truth, restored on boot. `backend/app/substrate/turso_http.py` is a minimal libSQL client over Turso's `/v2/pipeline` HTTP API (`requests` + `certifi`, pooled session, batched transactions). The same Turso DB also backs the real loss-data snapshot: `loss_db` reports its source as 'remote' (Turso), 'local' (baked `valmo.db`), or 'none'. If `TURSO_*` env is unset, everything degrades gracefully to plain local files — identical to prior local-only behavior.

Key points

- Single-service architecture: one FastAPI process serves both the JSON/SSE API and the compiled React SPA from the same origin in production; in local dev they run split (`uvicorn :8077`, `Vite :5190` with an `/api` proxy).
- Provider-swappable by design: OpenAI, Claude, and Gemini providers are all coded against one interface; the active choice and per-node fast/deep tier are pure config (`backend/config/models.yaml`). Today `provider=openai` with gpt-5.5 on both tiers, so the fast/deep split is present in the wiring but not exercised.

- Two SSE-streamed workflows (reactive chat, proactive monitor) plus two SSE authoring compilers; the browser reads them with fetch+ReadableStream (not native EventSource) so POST streaming and auth headers work.
- The reactive path is an LLM-driven bounded agentic tool-use loop (max 6 steps), not a hardcoded flow; deterministic code runs only inside tools, and apply_policy is the only path that can move money (checks + trust gate + adversarial verifier + idempotent write).
- Auth is server-enforced with role gates (viewer/author/approver; approver >= author) on a 12h signed token; only login and health are public.
- The active LLM path runs through the hackathon Bifrost gateway, and openai_provider.py contains substantial defensive code (browser-header spoofing to pass an Akamai WAF, aggressive retry of a ~70% 'virtual_key_not_found' flake) that is gateway-specific and would not be needed against a direct provider API.
- Deployment is one multi-stage Docker image on Render free tier; because that tier has no persistent disk, Turso (libSQL over HTTPS) is the durability layer for all authored/operational state and for the loss-data snapshot.
- Backend deps are deliberately lean: FastAPI, uvicorn, requests, pyyaml, python-dotenv, sse-starlette, plus document-extraction libs (pypdf, openpyxl, python-docx). No LLM vendor SDK is required for the active OpenAI path (the anthropic SDK is only needed if switching to Claude).

Status

Built and working now: the FastAPI backend, the React/Vite frontend (served standalone in dev and baked into the Docker image for prod), SSE streaming on chat + monitor + the two authoring compilers, the provider interface with all three providers implemented, the config-driven tiered registry, the OpenAI/Bifrost-gateway path with its WAF/retry handling, the multi-stage Dockerfile, the Render blueprint, and the Turso durable-state + loss-data layer (with graceful local-file fallback when Turso env is absent). Honest caveats to convey to leadership: (1) The active brain is OpenAI gpt-5.5 via a HACKATHON gateway (gateway-buildathon.ltl.sh), not a production Anthropic/OpenAI endpoint; the code contains gateway-specific workarounds and PRODUCTION_DELTA.md §K notes an unresolved organizer-infra issue where datacenter egress IPs can still be WAF-blocked. (2) The fast/deep tier split is real in configuration but a no-op today because both tiers map to the same model (gpt-5.5); the cost-saving 'cheap conversational tier' is designed, not yet running. (3) Claude and Gemini are fully coded and 'flip a YAML key' swappable, but Claude has not been exercised in the demo (no ANTHROPIC_API_KEY per the delta doc). (4) Some prose in PRODUCTION_DELTA.md §A still describes Gemini as the demo reasoning model; the authoritative, current config (backend/config/models.yaml) and README say OpenAI gpt-5.5 is active — treat the config as ground truth. (5) Render free tier means cold starts (~15 min idle sleep) and an ephemeral disk; production durability depends entirely on Turso being configured. Deployment/infra was verified only by reading config files, not by observing a live Render deploy in this session.

Key files

backend/app/main.py · backend/app/engine/conversation.py · backend/app/llm/registry.py ·
 backend/app/llm/base.py · backend/app/llm/openai_provider.py ·
 backend/app/llm/claude_provider.py · backend/app/llm/gemini_provider.py ·
 backend/config/models.yaml · backend/app/durable_state.py ·
 backend/app/substrate/turso_http.py · backend/app/substrate/loss_db.py ·
 backend/app/state_paths.py · frontend/src/lib/api.js · frontend/vite.config.js · (+6 more)

3. The Resolution Engine (agentic loop)

The Resolution Engine turns one captain message into one resolution using an LLM that drives the conversation and calls tools, rather than a hardcoded dialogue script. The model reads the message (in any language), decomposes it into concerns, and calls tools to look things up; but the only way money moves is the `apply_policy` tool, which runs deterministic checks in code, a trust gate, and an adversarial second-model verifier before any write. Every step is streamed to the browser as a live trace and also saved for replay and audit. The design mantra in the code is "the model proposes; policy disposes."

How it works

- 1. Capture the turn over a streaming connection** — `POST /api/chat` (backend/app/main.py) calls `conversation.handle_turn(conversation_id, captain_id, message, channel, attachments)` and wraps it in `EventSourceResponse(_sse(...))`. Every event the engine yields is pushed to the browser as a Server-Sent Event named 'trace' (JSON payload), and the stream ends with an 'end' event. `handle_turn` wraps the real loop (`_run_turn`) in `try/finally` so that, no matter what, the accumulated trace is persisted afterward.
- 2. Load session history and grounded context** — `session.py`'s in-memory `STORE.get_or_create` keeps the running chat history as a Gemini-style contents array keyed by `conversation_id` -- there is NO hand-rolled slot/state machine; the LLM sees full multi-turn context. The captain's message is appended. `captain_context.get_context(captain_id)` loads that captain's grounded records; an unknown captain yields an 'error' event and the turn stops. A 'capture' trace event is emitted (plus one for any attachments).
- 3. Select a tool-calling model and assemble the system prompt** — `llm_registry.for_node('classify')` returns a provider+model that must expose a `.chat()` (tool-calling) method (Gemini/Claude); if not, an 'error' event ends the turn. The system prompt = the long static `_SYSTEM` behavioural contract (decompose multi-intent messages; gather every signal; ground process answers in SOPs; never tell the captain to raise a ticket; and strict rules on when NOT to move money) PLUS `_blueprint_guidance()`, which additively injects any APPROVED Domain Blueprints (signals to gather, derivations so it never re-asks, decision branches, exact ask-if-missing prompts). Blueprint injection is defensive: any error or empty store returns an empty string and the loop behaves exactly as before.
- 4. Run the bounded agentic loop (max 6 steps)** — The loop iterates up to `MAX_STEPS = 6`. Each iteration calls `provider.chat(contents, tools=tools.DECLARATIONS)`. If the model returns NO function calls, its text IS the reply -> emit an 'explain' then a terminal 'reply' event and return. If it returns function call(s), execute each via `tools.dispatch`, append a `functionResponse` for EVERY call (even failures, so history never has a dangling tool call that would poison later turns), feed results back, and loop again. Multiple calls per step are expected (one `apply_policy` per AWB, one `escalate_case` per concern). If all 6 steps are exhausted, a safety fallback reply is emitted. A model-provider exception is caught and degrades gracefully to a fixed 'reasoning service unreachable' reply.
- 5. The five tools (tools.py DECLARATIONS + dispatch)** — `search_sops -> knowledge store.retrieve(k=8)`, returns snippets typed policy (rigid rule Valmo owns) vs procedure (functional/supply-chain), emits a 'knowledge' event. `get_captain_context -> that captain's profile/debits/losses/shipments/COD`, emits a 'ground' event. `run_data_query -> data_queries._run` runs ONE of five whitelisted named queries (`shipment_status`, `scan_history`, `payout_status`, `loss_summary`, `cod_status`) over the grounded context; the LLM only picks the query name, it never writes SQL; emits a 'query' event. `apply_policy -> the only money path (step 6)`. `escalate_case -> the only hand-to-human path (step 7)`.
- 6. apply_policy: the deterministic money spine** — `_apply_policy` calls `policy_exec.execute(disposition, context, entities)`. DATA-FIRST: if an AWB is supplied, it looks up the

REAL loss row in valmo.db (loss_db.get_loss_by_awb); the row's reason_l1 -- not the LLM -- maps to a disposition/policy, and real signals decide the outcome in _eval_real_loss: credit note issued -> already credited (respond); 0% / Meesho leg / no amount -> nothing was debited (respond); already revoked -> respond; a reverse_debit category WITH a reversal signal (facility in-scan OR attribution changed) within the cap -> reverse_debit; everything else -> ground the full record and escalate. If no AWB, it falls back to policy-by-disposition (hardstop_loss runs _exec_hardstop against the captain's ledger debit + Log10 scan trail + loss event: checks evidence-present, attributable-to-partner via connected_within_tat + hardstop_sop_followed, and within-cap; reverses only if all pass and the loss is NOT the partner's fault, else escalates). If an AWB/amount can't be matched (no live DB in this environment) it escalates honestly to Losses & Debits with the captain's details -- never a dead-end. Emits a 'policy' event carrying checks_run, action, evidence_trail, confidence.

- 7 **7. Trust gate, then adversarial verifier, then act-or-escalate** — trust/gate.py evaluate() applies hard limits OUTSIDE the prompt: required-evidence gate, money cap, a calibrated confidence threshold (CONFIDENCE_THRESHOLD = 0.80), and the Partner Constitution; it returns passed plus requires_adversarial_verify (true only when passed AND the action moves money). Emits a 'gate' event. If the gate passed and money is moving, trust/verifier.py runs an independent skeptic LLM that tries to REFUTE the decision; it fails CLOSED (agrees=False) if the verifier is unavailable, so an outage can never auto-approve money. Emits a 'verify' event. resolved = gate passed AND action != escalate AND verifier did not refute. If resolved: _act performs an idempotent write (idempotency_key) and emits an 'act' event, outcome resolved_in_conversation. Otherwise it escalates and deliberately relays an escalation-truthful line (NOT the unexecuted reversal reason) so the captain is never told money moved when it did not.
- 8 **8. escalate_case: worked hand-off to a functional team** — _escalate_case builds a fully-worked case from the captain's context plus every signal gathered (fe_id, hub, awb, amount, txn_id, when, attachments), routes by domain via _DOMAIN_TEAM to a named L2/L3 team, takes the ETA from I3.TEAM_SLA (single source of truth), appends an immutable concern with outcome 'escalated', and returns a reference id + team + ETA so the model can reassure the captain. If category is 'no_sop' it also logs a knowledge gap via kt.engine.log_gap (a 'learn' event) so a human can author the missing SOP. Emits an 'escalate' event.
- 9 **9. Compose the single reply and log the concern** — After tools return, the loop iterates once more; the model reads the tool results and writes ONE warm combined reply in its own words, constrained by the prompt to state only what a tool returned. The terminal 'reply' event carries the captain-facing text, decision_action, and concern_id. Money/escalation outcomes are appended to the Concern Log (ledger/concern_log.py append, adding logged_at + seq); a purely informational turn logs a lightweight concern via _log_info_concern (disposition 'conversation', outcome resolved_in_conversation).
- 10 **10. Persist the streamed trace as a replayable sidecar** — Every yielded event is accumulated in memory during the turn. In handle_turn's finally block, _persist_trace saves the full event list to traces.json (ledger/trace_log.py) keyed by concern_id -- once per concern the turn created (multi-intent turns create several, and the same worked trace attaches to each). Trace saving is wrapped in try/except so a save failure can NEVER break the live stream. This is what lets the Concern Log expand any resolution to show HOW it was reached and lets the Auditing Studio score it.

Key points

- Disposition (category) vs SOP scenario are two different granularities. A disposition is the concern's CATEGORY/theme (e.g. hardstop_loss, intransit_loss, damage, cod_pendency); it selects WHICH compiled ExecutablePolicy runs. A scenario is a specific branch inside that policy's decision tree (e.g. 'HS_1_1', set in _exec_hardstop when scans prove the auto-marking was Valmo-side). Category = which policy; scenario = which branch fired.

- How a disposition is chosen at run time: (a) DATA-first -- when an AWB resolves to a real loss row, `policies.reason_l1_to_disposition` maps the row's `reason_l1` to a disposition via a ~17-entry `_TAXONOMY` table (each row also carries the owning team, the action, and a cap); or (b) the LLM passes a disposition string on `apply_policy`. Only categories the snapshot data can decide (`hardstop`, `intransit`, `dual_scan_mismatch`, `debit_revoked`) are auto-actionable; the rest always escalate with a worked case.
- IMPORTANT honesty flag: `dispositions.py`'s `locate()` -- the similarity-search + `NOVEL_THRESHOLD` 'Continuous Problem Discovery' mechanism described in that module's docstring -- is `DEFINED` but is NOT called anywhere in the live engine (grep confirms no caller). Only `dispositions.catalogue()` is used, and only by the `/api/dispositions` endpoint to render the taxonomy view. The 'emergent, self-maintaining taxonomy / map-to-nearest-or-flag-NOVEL' behaviour is design-present, not exercised at resolution time.
- The streamed trace is a list of typed events, each shaped `{node, label, status, tier, detail, data}`. Node types seen: `capture`, `knowledge`, `ground`, `query`, `policy`, `gate`, `verify`, `act`, `escalate`, `learn`, `explain`, `reply`, `error`. tier is 'fast' or 'deep' -- the trust-spine steps (`policy`, `gate`, `verify`, `act`) are 'deep'. Delivered over SSE as event:'trace' and terminated by event:'end'; the identical list is persisted to the Trace Log.
- The money guarantee is layered and code-enforced, not prompt-enforced: deterministic policy checks (`policy_exec`) -> policy-as-code gate with a fixed 0.80 confidence threshold and hard money caps (`trust/gate`) -> adversarial verifier that fails closed (`trust/verifier`). The LLM can propose a reversal but cannot execute one, and the escalation path relays an escalation-truthful message so the captain is never falsely told money moved.
- Confidence values are heuristic constants set inside `policy_exec` (e.g. 0.9, 0.92, 0.93 for reverse/respond; 0.2-0.45 for escalate) and the gate threshold is a hardcoded 0.80. The gate docstring's claim that this is 'recalibrated continuously' describes intended production behaviour, not what runs here.
- Multi-intent handling is real: the loop allows several tool calls per step and tracks multiple concern ids per turn (`holder['concern_ids']`), so one message listing several AWBs or mixing FE-ID + payment concerns is worked and logged as separate concerns in the same turn, then answered in one combined reply.

Status

BUILT AND WORKING: the LLM tool-using loop (max 6 steps) with graceful degradation on LLM outage; all five tools; `apply_policy`'s deterministic policy execution against a local SQLite loss database; the trust gate (evidence/cap/confidence/Constitution) and the adversarial verifier (fail-closed); `escalate_case` with team routing, SLA-based ETA, and automatic knowledge-gap capture; the Concern Log and replayable Trace Log; SSE streaming; and additive Domain-Blueprint prompt injection. DEV/SNAPSHOT SUBSTRATE (not production): grounded data is a local SQLite file (backend/data/valmo.db, with a baked fallback) standing in for the docstrings' Metabase/Log10 platform; sessions are in-process (Redis in prod); the Concern Log and Trace Log are JSON files (Postgres in prod). DESIGN-ONLY / NOT WIRED: `dispositions.locate()` and its NOVEL/CPD similarity path are defined but never called by the engine -- disposition selection actually happens inside `policy_exec` (`reason_l1` mapping) or from the LLM-supplied disposition string. DELIBERATELY LIMITED: COD-pendency is intentionally NOT a money-moving policy (matches the real SOP -- L1 does not auto-clear pendency), so the `clear_pendency` action exists in `_act` but no seeded policy emits it; the only genuine autonomous money move is the `hardstop/wrong-debit` reversal. The engine requires a tool-calling provider (Gemini/Claude); without one the turn ends with an error event.

Key files

backend/app/engine/conversation.py · backend/app/engine/tools.py ·
backend/app/engine/policy_exec.py · backend/app/engine/session.py ·
backend/app/engine/dispositions.py · backend/app/engine/data_queries.py ·
backend/app/trust/gate.py · backend/app/trust/verifier.py ·
backend/app/knowledge/policies.py · backend/app/ledger/concern_log.py ·
backend/app/ledger/trace_log.py · backend/app/substrate/loss_db.py · backend/app/main.py

4. Trust Spine & Adversarial Verifier

The trust spine is a three-layer check that sits between the AI's proposed resolution and any real money movement. Layer one is the Partner Constitution — nine written rights (e.g. "no debit stands without evidence of partner fault") plus three action tiers. Layer two is a deterministic "trust gate" that enforces hard limits in plain code, outside the AI prompt: required evidence must be present, the amount must be within the policy cap, confidence must clear a calibrated threshold, and the Constitution must not be violated. Layer three, reached only for money-moving actions that clear the gate, is an adversarial verifier: a separate, deliberately skeptical LLM pass that tries to refute the decision, and money only moves if it agrees. All three are implemented and wired into the live resolution path (`backend/app/engine/tools.py::_apply_policy`), with a fail-closed rule so an unavailable verifier escalates to a human rather than auto-approving.

How it works

- 1 Engine proposes a decision** — `policy_exec.execute()` produces a decision dict — action (e.g. `reverse_debit`), `amount_inr`, `confidence`, an `evidence_trail`, `checks_run`, a reason, and the policy (with version and `resolution.cap_inr`). This is the input the trust spine judges.
- 2 Trust gate runs four deterministic checks (`gate.py::evaluate`)** — (1) Required-evidence gate: every item in `policy['required_evidence']` must appear in `decision['evidence_present']`, else it blocks. (2) Money cap: for money-moving actions {`reverse_debit`, `clear_pendency`, `credit`}, if `amount_inr > cap_inr` it blocks ('exceeds auto-action cap'). (3) Calibrated confidence: if `confidence < CONFIDENCE_THRESHOLD` (0.80) it blocks. (4) Partner Constitution via `check_constitution()`. Any non-empty block reason means the gate fails; it also returns `money_moving` and `requires_adversarial_verify = (passed AND money_moving)`.
- 3 Constitution check maps the decision to partner rights (`constitution.py::check_constitution`)** — It records which of the nine principles a decision UPHOLDS (a `reverse_debit` upholds good-faith/true-cause/auto-correction; an `evidence_trail` upholds transparency; a version upholds 'no silent policy changes'; amount within cap upholds proportionality). The ONLY condition that produces a Constitution VIOLATION is `amount_inr > cap_inr` ('must escalate, not auto-act'). `passed = (no violations)`.
- 4 Adversarial verifier runs only when the gate passes AND the action moves money** — `tools.py` calls `verifier.verify(decision, evidence_trail, reason)`. It resolves the 'adversarial_verify' node to the deep-tier model of the active provider (currently OpenAI gpt-5.5 per `config/models.yaml`) and sends a skeptical system prompt: refute if there is any reasonable doubt, default `refuted=true` when uncertain. The prompt also carries domain truth — Log10 physical scans (`INWARD_SCAN/MANIFEST_SCAN`) override an auto SLA loss-marking, so a scan-vs-loss-reason contradiction is the justification for reversal, not a reason to refute. It returns `{agrees, refuted, confidence, reason}`.
- 5 Verifier fails closed on error** — If the LLM call throws, `verify()` returns `agrees=False` with reason 'adversarial verifier unavailable — failing closed to human review'. An unavailable verifier can never auto-approve money; it forces escalation.
- 6 Pipeline decides act vs escalate (`tools.py::_apply_policy`)** — `resolved = verdict['passed'] AND action != 'escalate' AND verifier_agrees is not False`. If resolved, `_act()` performs the idempotent write (money actually moves); `outcome = resolved_in_conversation`. Otherwise the case is routed to the functional team (L2/L3) with a worked-case package, and the captain is told a fix couldn't be confirmed — the reversal reason is deliberately NOT relayed on escalation. Note `verifier_agrees=None` (verifier not required) does not block; only an explicit refute (`False`) does.

- 7 **Constitution is exposed for transparency** — GET /api/constitution (main.py) returns the nine PRINCIPLES and three TIERS, supporting the 'radical transparency' principle so the partner can see the rules the system binds itself to.

Key points

- Money-moving actions are a fixed set defined in code: {reverse_debit, clear_pendency, credit}. Only these trigger the adversarial verifier; other passing actions (e.g. read-only or escalate paths) resolve without it.
- Hard limits (evidence present, amount within cap, confidence ≥ 0.80) are enforced in ordinary Python OUTSIDE the LLM prompt — the model proposes, code disposes. The AI cannot talk its way past a cap or a missing-evidence block.
- The verifier's default posture is skepticism: 'default to refuted=true when uncertain.' Money moves only on affirmative agreement (agrees is not False).
- Fail-closed design: a verifier exception yields agrees=False, so infrastructure failure escalates to a human instead of silently approving a debit reversal.
- The Constitution encodes nine partner rights and three P&L tiers (tier1 correct-wrongs, tier2 fair-process, tier3 bounded-generosity); the tiers are descriptive metadata surfaced via the API, not enforcement logic in the gate.
- Provider/model routing is data-driven (config/models.yaml via llm/registry.for_node). Flipping one YAML key reroutes the verifier to Claude (claude-opus-4-8) or Gemini (gemini-2.5-pro) with no pipeline code change.

Status

Built and working: all three modules are fully implemented (no stubs) and are called on the live resolution path in tools.py::_apply_policy — gate first, then the adversarial verifier for money-moving actions, then the idempotent write. The fail-closed-on-error behavior and the escalate-vs-act branching are real code. The Constitution is served read-only at GET /api/constitution. Honest caveats: (1) 'Independent second-model skeptic' overstates the current config — the adversarial_verify node and the policy-reasoning node both resolve to the SAME deep-tier model id (today OpenAI gpt-5.5); the independence is an adversarial PROMPT and a separate call, not a different model. The architecture CAN route it to a different provider/model via YAML, but does not today. (2) The 'calibrated confidence threshold, recalibrated continuously in production' described in the docstring is not implemented — CONFIDENCE_THRESHOLD is a hardcoded constant of 0.80; there is no calibration/recalibration code. (3) check_constitution() is a lightweight rule-mapping, not an algorithmic proof over all nine principles: it tags which principles a decision upholds, but the only branch that can actually raise a violation (and thus block via the gate) is amount_inr exceeding the policy cap — which largely duplicates the gate's own cap check (constitution applies it regardless of the money_moving flag, so it is marginally broader). The other eight principles are asserted/recorded, not independently verified in code."

Key files

backend/app/trust/constitution.py · backend/app/trust/gate.py ·
backend/app/trust/verifier.py · backend/app/engine/tools.py · backend/app/llm/registry.py ·
backend/config/models.yaml · backend/app/main.py

5. Knowledge Model & SOP Compiler

The platform stores partner-support know-how as structured knowledge on three levels: Domain (a fixed queue like losses or payments) -> Disposition (a snake_case concern category, e.g. hardstop_loss) -> SOP (one specific scenario/rule). The "SOP Compiler" is the authoring feature that takes a plain-language SOP written by a functional team and, in a single LLM pass, converts it into a structured "ExecutablePolicy" object with an explicit trigger, evidence list, yes/no checks, a resolution + money cap, an escalation route, the exact captain-facing reply and links, and the partner rights it protects. The core design principle in the code is "understand once (LLM at authoring time) -> execute every time (deterministic code at resolution time)": the compiler only structures the SOP, it never decides a live case. Compilation is streamed as visible stages and is wrapped in three guardrails - structural gap detection, fidelity checking (did the machine drop any specifics from the source text), and duplicate detection - plus an advisory governance-conformance score.

How it works

- 1 Three-level knowledge model is enforced by the compiler prompt —** backend/app/knowledge/sop_compiler.py's _PROMPT requires the LLM to return domain (one of a fixed enum: losses | payments | fe_id | cod_cash | consumables | orders | other), disposition (the concern CATEGORY as a snake_case key), and title (the specific scenario name). To stop the taxonomy sprouting near-duplicate categories, known_dispositions() pulls the distinct disposition keys already in the store and injects them into the prompt with an instruction to REUSE an existing key if one fits.
- 2 One LLM pass compiles plain text into an ExecutablePolicy —** compile_sop(sop_text) resolves the model via llm_registry.for_node('sop_compile'), which config/models.yaml maps to the 'deep' tier. It sends the _SYSTEM + _PROMPT, parses the JSON reply, and returns (policy, meta) where meta carries model/provider/token counts/latency. The returned JSON shape includes id, title, domain, disposition, version, trigger{keywords, preconditions}, required_evidence[], checks[{id, description, reads, expect}], resolution{action, params{idempotent}, cap_inr}, captain_reply, links{label: url}, escalation{team, handover}, and partner_rights[].
- 3 The prompt forces operational specifics to be copied verbatim —** The prompt explicitly tells the model NOT to abstract away specifics: every form/tracker URL must go into links (and be referenced from captain_reply), every named person/POC/team and their order must go into escalation.handover, and any 'reply to the captain / fill this form' instruction must go into captain_reply. partner_rights may only cite from a fixed list of nine Partner Constitution principles named in the prompt (presumption of good faith, true-cause attribution, radical transparency, guaranteed SLAs, auto error-correction, proportionality + downside caps, right to appeal + a human, consistency, no silent policy changes).
- 4 Compilation is streamed as visible stages —** compile_sop_streamed() yields stages in order for the UI to animate: understand -> trigger -> required_evidence -> checks -> resolution (with cap_inr) -> escalation -> partner_rights -> (optionally) conformance -> done. Each middle stage carries the real extracted slice of the policy. The final 'done' event carries the full policy plus gaps, meta, similar_sops, and conformance.
- 5 Guardrail 1 - structural gap detection —** detect_policy_gaps() flags missing decision logic as {severity, where, message}: no trigger keywords (retrieval won't fire), no required_evidence, no checks (high - no yes/no test), a check whose description is blank, a check that reads evidence NOT listed in required_evidence (high - engine can't gather it), no resolution.action, and no escalation.team (high - every SOP needs a safe fallback owner).

- 6 **Guardrail 2 - fidelity check against the source text** — `detect_fidelity_gaps(source_text, policy)` compares the ORIGINAL SOP text to the compiled JSON to catch things the machine dropped: URLs present in the source but absent from the policy (high - captains won't get the form/tracker), a source that says 'fill the form / reply / use template' but an empty `captain_reply` (high - the bot won't relay it), and a multi-step relay (source contains an arrow or 'step N') flattened into a handover string shorter than 40 chars (warn). This is what the code calls turning 'hope it got it right' into 'it shows what it missed'.
- 7 **Guardrail 3 - dedup / similarity** — `find_similar_sops(title, text)` scans already-compiled SOPs and scores each with the max of a stopword-filtered token-set Jaccard overlap and a difflib title ratio (stdlib only, no new dependencies). Matches scoring ≥ 0.4 are returned best-first, capped at 3, as a non-blocking warning so the author sees overlapping existing SOPs before adding another.
- 8 **Advisory governance conformance** — During the stream the compiler lazily imports `knowledge/governance.py` and calls `check_conformance(policy)`, returning `{band, conformant, findings}` scored against the org's Governance Framework (owner present, money cap, partner rights, idempotency, band). It is defensive (wrapped so it never breaks the compile stream) and advisory - it never blocks the write.
- 9 **Persisting a compiled SOP as retrievable knowledge** — `_persist_sop()` (via `approve_sop / save_sop_draft`) renders the structured policy into a canonical plain-language 'knowledge' statement and stores it in the durable `kt_queue.json` as a KT entry tagged `compiled_sop=True`, with the full policy kept under the 'policy' key. Writes are serialized by a threading lock with an atomic read-modify-write; if the store is unreadable it REFUSES to write (so it can't clobber authored SOPs). Editing in place reuses the same id; a new save drops any earlier DRAFT of the same disposition so drafts don't pile up. Drafts do NOT enter the corpus; `approve_sop` sets `status=approved` and calls `store.reload()`.
- 10 **The retrieval corpus and how approved SOPs enter it** — `backend/app/knowledge/store.py` loads `corpus.json` (a baked ingested snapshot, 467 chunks) and MERGES every approved entry from `kt_queue.json` so newly-approved SOPs are instantly retrievable without re-ingest. `retrieve(query, k)` is a deterministic lexical scorer: token overlap after Hinglish alias normalization (`galat->wrong`, `nuksan->loss`, etc.) and light stemming, with boosts for tag matches (x2), title/tag hits (x1.5), queue match, and authoritative sop/scenario chunks (+0.5), a length penalty for very long chunks, and a relevance cutoff so only chunks near the top score are returned. `reload()` clears the `lru_cache` so approvals take effect live.
- 11 **Seeded knowledge on first boot** — `backend/app/kt/engine.py` bakes seed SOPs so the library and corpus are non-empty at startup: a hand-structured COD Shortfall ExecutablePolicy (from the domain owner's step-by-step doc, complete with a real Google Form link and the Form -> Syed -> Cost Ops -> DAP POC relay) plus the SOPs parsed from the 'SOP Redressal Tracker' sheet in `seed_sops.json`. `_load()` reconciles seeds by stable id additively: a missing seed is added, a pristine seed whose baked content changed is refreshed, and any team-edited or authored entry is left untouched (user edits always win).

Key points

- The compiler's job is structuring, not deciding: `_SYSTEM` says 'You do NOT resolve any case - you only structure the SOP so a deterministic engine can execute it every time. Money caps must be explicit. Every check must name the evidence it reads.'
- There are TWO distinct policy stores. The SOP Compiler writes compiled SOPs into the `kt_queue.json` knowledge store (the retrieval corpus). The separately code-seeded registry in `policies.py` (hero `hardstop_reversal` + one auto-generated policy per `reason_l1` loss category) is what `policies.py` describes as feeding the deterministic money-move engine. Within the files reviewed,

compiling/approving an SOP makes it retrievable knowledge that steers the grounded reply; it does not itself add a new entry to the policies.py registry.

- 'Deep tier' is a routing label, not a fixed model. config/models.yaml currently sets provider: openai with both fast and deep tiers mapping to gpt-5.5, so sop_compile runs on gpt-5.5 today. The sop_compiler.py docstring's 'Opus tier' reflects the Claude alternative (tiers.claude.deep = claude-opus-4-8) that the same registry would route to if provider were flipped to claude - no pipeline code change needed.
- The ExecutablePolicy dataclass in policies.py (id, disposition, version, trigger, required_evidence, checks, resolution, escalation, partner_rights, required_inputs, identify_any) is a slightly narrower shape than the compiler's JSON, which additionally carries title, domain, captain_reply, and links. The compiler's richer fields are preserved on the stored KT entry's 'policy' object and rendered into the retrievable knowledge text.
- captain_reply + links are treated as first-class: render_captain_reply() (in kt/engine.py) fills {form}/{tracker} placeholders in the reply from policy.links so the exact captain-facing message (and its real URLs) is preserved rather than abstracted away.
- Check shapes differ between seed and compiled SOPs: the compiler emits checks as {id, description, reads, expect}, while the hand-seeded COD policy uses {description, source}. Only the description is rendered into the retrievable knowledge statement, so both work at retrieval time.
- Gaps are surfaced at compile/save time inline (severity warn vs high), not in a separate queue; the Authoring Guide states 'Approve & go live' stays disabled until red (blocking) gaps are cleared - this section's code produces the gap list, and the enforcement lives in the UI/authoring layer.
- A minor inconsistency: kt/engine.py's comment says '64 SOPs' from the tracker sheet, but seed_sops.json currently contains 68 entries.

Status

BUILT AND WORKING: single-pass LLM compilation (compile_sop) and its streamed stages (compile_sop_streamed); all three guardrails (detect_policy_gaps, detect_fidelity_gaps, find_similar_sops) are fully implemented in plain Python; persistence with draft/approve/delete, atomic locked writes, in-place edit, and a refuse-to-write-if-unreadable safety; corpus merge of approved KT + live reload; the lexical retrieval scorer with alias normalization and relevance cutoff; and the baked seeds (COD Shortfall + the tracker seed_sops.json, reconciled additively by id). Governance conformance scoring is wired into the compile stream but strictly advisory (lazy-imported, wrapped so it never blocks or breaks compilation). DESIGN-ONLY / FUTURE (explicit in the code): store.py's docstring states the lexical scorer is a deterministic stand-in and that Voyage/BGE embeddings + pgvector 'plug in' in production behind the same retrieve() contract - the embeddings path is NOT built. kt/engine.py's docstring lists 'learn the SOP-writing pattern -> auto-draft SOPs' as 'documented, not built.'

CONFIGURATION CAVEAT: the compiler docstring calls this the Opus tier, but the active config runs gpt-5.5 (OpenAI) for sop_compile; Opus is the ready-but-inactive Claude alternative. NUANCE FOR LEADERSHIP: 'ExecutablePolicy' names the structured, execution-ready shape the compiler produces and stores as searchable knowledge; the autonomous money-moving execution engine (per policies.py) reads a separate, code-seeded policy registry, so authoring an SOP here enriches the reasoning/reply layer rather than directly minting a new autonomous payout rule.

Key files

backend/app/knowledge/sop_compiler.py · backend/app/knowledge/policies.py ·
backend/app/knowledge/store.py · backend/app/kt/engine.py ·
backend/app/knowledge/governance.py · backend/app/llm/registry.py ·
backend/config/models.yaml · backend/data/knowledge/corpus.json ·
backend/data/knowledge/seed_sops.json · docs/AUTHORING_GUIDE.md

6. Domain Brains & the Authoring Studio

The Authoring Studio (the ``AuthoringStudio`` view in ``frontend/src/pages/SupportCommand.jsx``) is where a domain owner teaches the engine in plain language, in two shapes: a Domain "Brain" (a Blueprint — how to think about a whole domain such as losses or payments) and individual SOPs (specific rulebooks). The author types a free-text walkthrough — or uploads a real Excel/Word/PDF/CSV artifact that is extracted to text first — clicks Compile, and an LLM structures it into a fixed-shape object that streams into the right-hand pane stage by stage. As it compiles, "gaps" (things that would leave the machine not knowing what to ask or how to decide) surface as inline amber/red chips; the author edits the structured object directly, then Queues it (draft) and, if they are an approver, Approves it to go live. Approved Brains are injected additively into the live resolution engine's system prompt — they steer it but never replace the SOP/policy/tool path.

How it works

- 1 One studio, two modes** — AuthoringStudio has a toggle for "Domain Brain" vs "SOP" (state ``mode`` = `brain|sop`, default `brain`). Left pane: raw-text box + an optional domain picker + a Compile button. Right pane: the streamed structuring, then an editable structured view (BlueprintEditor or SopEditor), gap banner, and Queue/Approve actions. Both modes share the same compile→edit→queue→approve pipeline; only the shape of the object and the endpoints differ.
- 2 File-upload ingestion (Excel/Word/PDF/CSV/txt → text)** — The Upload-file button accepts `.xlsx/.xlsm/.docx/.pdf/.csv/.txt/.md` and POSTs the file to `/api/sop/extract`. Backend ``_extract_text`` (backend/app/main.py) does best-effort extraction: PDF via `pypdf`, `.xlsx` via `openpyxl` (each sheet's rows flattened to ``a | b | c`` lines), `.docx` via `python-docx` (paragraphs + table rows flattened the same way), everything else decoded as UTF-8. It enforces a ~12MB cap (413), rejects legacy `.xls/.doc` with a re-save message, and 400s on empty content. This step runs NO LLM and writes NO state — it only fills the raw-text box so the author reviews/edits the text, then Compiles through the normal pipeline.
- 3 Compile a Brain (LLM structuring, streamed)** — Compile in brain mode calls `compileBlueprintStream` → POST `/api/blueprint/compile` → ``blueprints.compile_blueprint_streamed``. It calls the LLM on the ``sop_compile`` node (deep tier, same node the SOP compiler uses) with a fixed system prompt ("Domain-Brain Compiler") and a template that forces EXACTLY this JSON shape: `domain`, `label`, `signals`, `derivations`, `lookups`, `decision`, `ask_if_missing`, `escalation_team`, `proactive`. Domain is either the one the author locked, or — if left on "auto-detect" — inferred by the LLM from a fixed 7-value enum. The result streams as SSE stages (`understand` → `signals` → `derivations` → `lookups` → `decision` → `ask_if_missing` → `done`); the frontend animates each stage (BlueprintCompileAnimation) and the final ``done`` event carries `{blueprint, gaps, meta, existing_brain}`.
- 4 What a Brain actually captures** — `signals` = the clues the owner reads, each tagged with a source (`message/profile/image/either`); `derivations` = rules that resolve several clues to ONE canonical key so the captain is never re-asked (e.g. `hub_code + debit_note_no` → `awb`); `lookups` = which fields to fetch from a data source once you hold a given key; `decision` = yes/no conditions each mapping to an action (`reverse | inform_educate | escalate | respond`); `ask_if_missing` = the exact prompt (in the captain's Hinglish) to send when a needed key is missing and can't be derived; `escalation_team` and `proactive` round it out. An approved "Losses & Debits" Brain ships as a seed example.

- 5 **Inline gap detection at authoring time** — ``blueprints.detect_gaps(blueprint)`` runs on the compiled/edited object and returns {severity, where, message} items. Rules: a signal with a blank/unknown source (warn); a decision branch with an empty/vague condition — checked against a hard-coded vague-word list and a <4-char length test (warn); a lookup ``when_have`` key that no signal or derivation produces (high); a needed lookup key that can't be derived AND has no `ask_if_missing` fallback (high); an `ask_if_missing` entry with an empty prompt (warn); and NO escalate branch anywhere (high). Detection is heuristic/string-based, not semantic.
- 6 **Gaps rendered inline; high gaps block go-live** — The ``done`` event's gaps are shown as chips: warn=amber, high=red, placed next to the exact section/row they point at (gapsFor matches by ``where`` path). A summary banner counts total gaps and how many are "blocking". The "Approve & go live" button is disabled while any high-severity gap remains (``highGaps > 0``).
- 7 **Edit the structured Brain in place** — BlueprintEditor renders each section as editable add/remove/change rows (Field/RowCard/AddRow primitives). The domain is a confirmable dropdown pre-filled from the inferred/explicit value, so the author confirms or overrides the machine's domain guess after compiling.
- 8 **Dedup guardrail — one Brain per domain** — Brains are keyed one-per-domain. ``existing_brain(domain)`` returns a compact summary if a Brain already exists; the UI shows a non-blocking amber banner ("A Brain for X already exists — you're editing it, or open the existing one") with an "Open existing" button. Saving a Brain for an existing domain overwrites that entry (it is an edit, not a second record).
- 9 **Queue (draft) vs Approve (go live), gated by role** — Queue changes → saveBlueprint → POST /api/blueprint/save stores it with status=draft (open to authors). Approve & go live (approvers only — gated by ``isApprover``) saves then calls approveBlueprint → POST /api/blueprint/approve, which sets status=approved, stamps updated_at, and calls ``store.reload()`` to refresh the retrieval corpus caches. Non-approvers see only "Queue for approval" with a lock hint. Storage is a JSON file in a durable state dir (blueprints.json) that survives redeploys; the store seeds the approved Losses Brain on first run if empty.
- 10 **How an approved Brain steers the live engine** — In backend/app/engine/conversation.py, ``_blueprint_guidance()`` turns every APPROVED Blueprint into a system-prompt addendum appended to the engine's base prompt: the signals to gather, the derivations ("so never re-ask if you can derive it"), the decision branches (If <condition> → <action>), the ask_if_missing prompts verbatim, and the escalation owner. It is purely additive and defensive — wrapped in try/except, and an empty store or any error returns "" so the loop behaves exactly as before. It does not replace the SOP/policy/tool path, and the money guarantees still run through tools.
- 11 **The Knowledge Base / Authored library** — Below the editor, the "Authored library" lists two columns fetched on mount: Domain Brains (GET /api/blueprints, each returned with freshly recomputed gaps) showing label, domain key, signal/branch counts, escalation team, a status pill (draft vs "approved · live"), and a gap count; and Compiled SOPs (derived from KT entries that carry a compiled_sop, via GET /api/kt) showing title, id, check count, and status. Clicking any card loads that object back into the editor for editing.
- 12 **SOP mode (parallel pipeline)** — SOP mode uses compileSopStream / SopEditor / saveSopDraft / approveSop and adds a Governance conformance gate: before an SOP goes live, checkSopConformance re-scores it and any HIGH-severity governance finding triggers a confirm dialog (advisory — the author can override). Per the in-product "How this works" help, SOPs are NOT domain-keyed, whereas Brains are one-per-domain.

Key points

- TWO distinct gap detectors — do not conflate them. (1) ``blueprints.detect_gaps`` runs INLINE in the Authoring Studio at creation/edit time and is fully wired into the UI. (2) ``gaps.py` `detect()`` is a separate CORPUS-WIDE scan (procedures that never state "required from the captain", plus auto-captured escalation gaps) exposed at GET `/api/sop/gaps`.
- The ``/api/sop/gaps`` endpoint (`gaps.py`) has NO current frontend consumer — grep of `frontend/src` finds no call. Its docstring names a "Support Console authoring panel" that is not present in `SupportCommand.jsx`. The auto-captured gaps it would surface DO appear separately in the Learning Queue tab, but via the raw KT queue (`/api/kt`), tagged AUTO-GAP with a `hit_count`, not through `gaps.detect()`.
- The mechanism by which the engine "follows" an approved Brain is prompt injection (additive system-prompt text via `_blueprint_guidance`), not a hard rule engine or code change. Approval reloads the corpus and the next turn's prompt includes the Brain.
- `_blueprint_guidance` injects signals, derivations, decision branches, `ask_if_missing` prompts, and `escalation_team` — but NOT the Brain's ``lookups`` or ``proactive`` fields. Those are authored and stored but not surfaced to the live engine anywhere in the code reviewed.
- Domain enum is fixed to 7 values: losses, payments, `fe_id`, `cod_cash`, consumables, orders, other. ``cash_cod`` (the engine tool's spelling) is accepted on read and normalized to ``cod_cash``.
- Roles enforced: authoring/queuing a draft is open to authors; making anything go live requires an approver (`isApprover`). Additionally the UI blocks go-live while any high-severity gap is unresolved.
- Gap detection is heuristic and string-based (a vague-word blocklist, a min-length check, set-membership on which keys are produced) — it is not a semantic or LLM check.
- File-upload extraction is text-only and best-effort: tables are flattened to ``a | b | c`` rows, ~12MB cap, legacy `.xls/.doc` rejected, 400 on empty. It runs no LLM and writes no state — it only prefills the text box for the author to review before compiling.
- Persistence is JSON files in a durable state dir (`blueprints.json`, `kt_queue.json`) that survive redeloys; the store auto-seeds an approved "Losses & Debits" Brain on first run.

Status

BUILT AND WORKING: Domain-Brain compilation (LLM on the `sop_compile` node) with a fixed JSON shape and streamed SSE stages; the animated + editable structured view; inline gap detection with amber/red chips and a hard block on go-live while high gaps remain; the one-Brain-per-domain dedup banner; draft-save and approver-only approve with corpus reload; live-engine consumption via an additive system-prompt addendum (defensive, never breaks the loop); the Authored library listing Brains and compiled SOPs with click-to-load; and file-upload extraction for `.xlsx/.xslm/.docx/.pdf` and UTF-8 `text/csv/md`. A ready "Losses & Debits" Brain ships as an approved seed. The SOP mode runs the same pipeline plus a Governance conformance gate.

NOT FULLY WIRED / DESIGN-ONLY: The corpus-wide SOP gap scanner in `gaps.py` (GET `/api/sop/gaps`) is implemented on the backend but is not called by any current frontend view — the "Support Console authoring panel" its docstring references does not exist in the code reviewed (the inline Blueprint gap detector is what the Studio actually uses). A Brain's ``lookups`` and ``proactive`` fields are authored, stored, and shown in the editor, but they are NOT injected into the live engine's guidance, and there is no proactive-monitoring pipeline consuming ``proactive`` in the code reviewed — it is currently guidance text only. Captured/auto-gap surfacing depends on the engine writing `auto_gap` KT entries on no-SOP escalations (`kt/engine.py`), which then appear in the Learning Queue rather than through `gaps.detect()`. Gap detection is heuristic/string-based, not semantic, so it catches structural omissions (blank source, missing escalate branch, un-producible lookup key) but will not catch a plausible-looking-but-wrong condition or prompt.

Key files

backend/app/knowledge/blueprints.py · backend/app/knowledge/gaps.py ·
frontend/src/pages/SupportCommand.jsx · backend/app/main.py ·
backend/app/engine/conversation.py · frontend/src/lib/api.js ·
frontend/src/components/BlueprintCompileAnimation.jsx · backend/app/kt/engine.py

7. Governance Framework & Conformance Loop

The Governance Framework is an editable model of how the org identifies, scores, prioritises, and assigns accountability for partner problems. It lives entirely as one JSON document (not code), so the captain-governance team can rewrite every part — signal sources, scoring dimensions, bands, metrics, owners, and a catalogue of prioritised problems — in the Auditing Studio with zero code change, or upload a governance document and have an LLM structure it. The NEW conformance loop turns that framework into an active check: every SOP compiled in the Authoring Studio is scored against the framework's mandates (an accountable owner, a money ceiling on automated payouts, a binding partner right, idempotent money moves, and a priority band). This is honestly advisory, not enforced: it surfaces findings and, at approve time, a confirmation dialog the author can override — it does not hard-block a go-live. Critically, the framework that ships today is a clearly labelled PLACEHOLDER (Meesho's Seller Governance OS); the real Valmo captain-governance model has not yet been authored.

How it works

- 1 The framework is stored as one versioned JSON document —**
backend/app/knowledge/governance.py keeps the model in governance_framework.json under the durable-state directory (survives redeloys; default backend/data). The shape includes: name, version, status ('draft'/'approved'), objective, note, signal_sources (each tagged reactive/proactive), dimensions (key, label, scale_min/max, sub_factors, and an optional ordinal 'ladder'), a combine rule ('multiply'/'weighted_sum') + a formula string, bands (label/meaning/action), band_movement rules, metrics, an accountability block (owners_by, sla_hours, score_def), and an items catalogue of prioritised problems. get()/save(draft)/approve() manage it behind a thread lock, and _coerce() normalises any author- or LLM-supplied object into this shape — filling defaults, guaranteeing every list key exists, never dropping author content.
- 2 The seed framework is a labelled PLACEHOLDER —** _seed() returns a framework literally named 'Seller Governance OS — PLACEHOLDER (replace with Valmo captain governance)', with a note field (_SEED_NOTE) stating it mirrors Meesho's Seller Governance OS and every field is editable with zero code change. Its three problem items each carry "example": true. The on-disk seed exists at version 1, status 'approved'. It is illustrative data meant to be replaced, not a real Valmo model.
- 3 Editing and publishing go through draft → approve with versioning —** save(framework) coerces the edit, marks status='draft', and preserves the current version number. approve() coerces, sets status='approved', and bumps version by 1. API routes in main.py: GET /api/framework (any authenticated user), POST /api/framework to save a draft (author role), POST /api/framework/approve to publish (approver role only). The frontend GovernanceFramework page renders an at-a-glance overview plus fully editable rows for every part.
- 4 Uploading a document structures it into a draft via the SOP compiler's LLM path —** POST /api/framework/upload (author role) extracts text from the file (_extract_text handles PDF via pypdf, and utf-8 for csv/md/txt), then structure_framework_from_text() calls the LLM using the exact same mechanism the SOP compiler uses — llm_registry.for_node('sop_compile') + json_mode + _parse_json — against a strict JSON schema in _PROMPT/_SYSTEM. It does one retry with a stricter re-ask if the first result has no dimensions or bands, then coerces the result. It ALWAYS returns a DRAFT slotted at the current version; it never auto-approves — a human reviews before publishing.
- 5 Money-moving detection is a deliberately specific regex —** _moves_money(resolution) returns True if the resolution declares a ■ cap (cap_inr is not None) OR its action text matches _MONEY_MOVE_RE — a verb list of reverse/refund/credit/compensate/waive/reimburse/adjust/settle/disburse and 'pay out/the/back/■/rs'. It is written to fire on actual money verbs but NOT on the noun 'payout' (e.g. 'explain the payout timeline').

The comments note this errs toward false-positives (safer to demand a cap on an info SOP than to certify an unbounded payout).

- 6 **classify_band maps an SOP to a framework band** — `classify_band(policy, fw)` uses an explicit `policy.priority` that matches a framework band label if present; otherwise it infers — if the policy moves money OR contains hard-stop signals ('hardstop', 'deactivat', 'block', 'fe id', 'codb'), it picks the first (most severe) band, else the second band. It returns {band, action (the framework's mandate for that band), inferred (True when it had to guess)}.
- 7 **check_conformance scores a compiled policy against five framework mandates** — `check_conformance(policy, fw)` produces findings: (1) HIGH — no `escalation.team` = no accountable owner; (2) HIGH — moves money but `cap_inr` is None = unbounded automated money move; (3) WARN — no `partner_rights` cited; (4) WARN — money move not marked `resolution.params.idempotent` (retry could double-pay); (5) INFO — band was inferred, confirm it. It returns {band, band_action, conformant, findings, mandates}. `conformant` is False ONLY when a HIGH finding exists (missing owner or unbounded money); WARN and INFO never flip it. The code comments state it is advisory by design, at parity with `detect_policy_gaps/detect_fidelity_gaps`.
- 8 **Conformance is wired into the compile stream and stamped on stored SOPs** — In `sop_compiler.compile_sop_streamed()`, after the resolution/escalation/partner-rights stages, it calls `governance.check_conformance(policy)` inside a try/except (so it can never break the SSE stream), emits a 'conformance' stage event with band/conformant/findings, and includes the full conformance object in the final 'done' event (served by POST `/api/sop/compile`). Separately, `_sop_entry()` stamps `entry['conformance'] = check_conformance(p)` on every persisted SOP — draft or approved — so the Knowledge Base can display its band and whether it conforms. POST `/api/sop/conformance` (author role) re-scores a policy on demand after an edit.
- 9 **The approve-time gate is a client-side advisory confirm, not a server block** — The gate lives in the frontend `approve()` handler in `SupportCommand.jsx`: before approving an SOP it re-scores the current policy via `checkSopConformance`, filters findings with severity 'high', and if any exist shows a `window.confirm()` listing them ('Approve and go live anyway?'). Cancelling aborts; confirming proceeds to `approveSop()`. The backend POST `/api/sop/approve` (approver role) does NOT check conformance — it just persists and reloads the retrieval corpus. So the 'gate' is a soft, overridable confirmation on the client, not enforced governance on the server.

Key points

- The framework is DATA, not code: one JSON file, fully editable via the Auditing Studio (or LLM-structured from an uploaded doc) with zero code change. Draft→approve with an incrementing version number.
- The shipped framework is an explicitly labelled PLACEHOLDER named 'Seller Governance OS — PLACEHOLDER (replace with Valmo captain governance)', with example items flagged "example": true. The real Valmo captain-governance model has not been authored yet.
- Conformance scoring is real, working code: five mandate checks (owner, money cap, partner right, idempotency, band) with severity levels, plus regex-based money-move detection and band classification.
- Conformance is advisory throughout. `conformant=False` only on a HIGH finding (missing owner OR unbounded money move); WARN/INFO never block. The code comments explicitly call it advisory (parity with the existing gap detectors).
- The approve-time gate is a frontend `window.confirm` the author can override — the backend `approve` route does NOT enforce conformance. Describing it as a hard block would overstate it.

- Uploaded-document structuring reuses the exact SOP-compiler LLM mechanism (for_node('sop_compile') + json_mode + _parse_json) and always returns a draft for human review — it never auto-approves.
- The code itself flags the conformance rules/thresholds as 'intentionally conservative' INFRA to be fine-tuned as the Valmo governance framework matures.
- Role separation: viewing = any authenticated user; saving drafts/uploading/on-demand conformance = 'author' role; publishing the framework or approving an SOP = 'approver' role.
- Note: an unrelated _governance_placeholder in l3/platform.py (the L3 escalation-ladder outcome) is a different concept and is not part of this framework.

Status

BUILT AND WORKING: the JSON store with versioning and draft/approve lifecycle (get/save/approve, _coerce); all four framework API routes (GET /api/framework, POST /api/framework, /api/framework/upload, /api/framework/approve) with author/approver role gating; the LLM structure-from-text upload path (always returns a draft, one retry, human-review-before-approve); the conformance scoring engine (check_conformance, classify_band, _moves_money, the money-verb regex) with its five mandate checks; wiring into the compile SSE stream (a 'conformance' stage + full object in the 'done' event); conformance stamped on every persisted SOP; the on-demand POST /api/sop/conformance route; and the frontend editor, overview, compile animation, and the approve-time confirm dialog. PLACEHOLDER / NOT REAL CONTENT: the seed framework itself is explicitly labelled a placeholder mirroring Meesho's Seller Governance OS — the actual Valmo captain-governance model (dimensions, bands, metrics, owners, real problem items) has not been authored. The three seed problem items are marked as examples. ADVISORY, NOT ENFORCED (do not overstate): conformance never hard-blocks a go-live. conformant flips to False only on a HIGH finding, and even then the only 'gate' is a client-side window.confirm in the frontend that the author can override with a reason of record; the backend /api/sop/approve does not check conformance at all. The code comments themselves describe the rules/thresholds as intentionally conservative infrastructure to be tuned as the framework matures. Minor: governance.py imports state_path but does not use it (unused import; no functional impact).

Key files

```
/Users/pranav.akella/PSP/backend/app/knowledge/governance.py ·
/Users/pranav.akella/PSP/backend/app/knowledge/sop_compiler.py ·
/Users/pranav.akella/PSP/backend/app/main.py ·
/Users/pranav.akella/PSP/backend/data/governance_framework.json ·
/Users/pranav.akella/PSP/frontend/src/pages/SupportCommand.jsx ·
/Users/pranav.akella/PSP/frontend/src/components/PolicyCompileAnimation.jsx ·
/Users/pranav.akella/PSP/frontend/src/lib/api.js
```

8. L3 Escalation Platform

The L3 Escalation Platform (`backend/app/l3/platform.py`) is the accountable inbox where concerns that the front-line agent could not resolve land for a human functional team to work. It is deliberately not a chatbot: each escalated case arrives with its fully-worked context (evidence trail, the captain's original reply, entities like AWB/FE-ID/amount, attachments), an SLA timer that starts the moment it was logged, a per-team escalation ladder, and a resolve action that closes the loop back to the captain. The whole platform is a read-and-append view over the single append-only Concern Log — there is no separate L3 database. One piece is intentionally not real yet: the governance severity scorer is an explicit placeholder that the product owner has flagged for later replacement.

How it works

- Escalations land in the shared Concern Log, not a separate store** — There is no L3-specific database. When the agent gives up, one of two producers in `engine/tools.py` writes a concern record with `outcome="escalated"` and `action_taken="escalate"` into the append-only `concern_log` (a JSON file, Postgres in production per the BRD). (a) The `escalate_case` tool (`_escalate_case`) stamps `escalation_team` directly from a `domain→team` map (`_DOMAIN_TEAM`, e.g. `payments → "Payments (L2)"`) and attaches a worked evidence trail, captain-provided entities and any attachments. (b) The `money/policy` path (`_apply_policy`) stamps `escalation_team` from the matched policy's `escalation.team`, defaulting to `"Losses & Debits (L2)"`.
- The inbox is computed live from the log every time it is called** — `inbox()` reads all concerns (newest-first), keeps only those with `outcome=="escalated"`, and drops any whose `id` already appears as another concern's `resolves_concern_id` (i.e. cases that have been resolved-back) unless `include_resolved=True`. So the active queue is derived, not stored — resolving a case makes it fall out of the queue automatically.
- SLA and breach status are derived per team from a hardcoded table** — `TEAM_SLA` maps 12 named teams to an `sla_hours` value (12/24/36/48h) and a ladder list; unknown teams fall back to `DEFAULT_SLA` (48h). For each item, `age_hours` = hours since `logged_at` (via `_hours_since`), and `breached` = `age_hours > sla_hours`. `_team_of` resolves the team: use the stamped `escalation_team` if present, else derive it from the concern's policy. The module comment calls this table "config, not code (BRD §15.2)" but it is currently a plain Python dict in the source file. This same table is the single source of ETA truth — `engine/tools.py` imports it so the ETA it promises the captain matches the SLA the inbox enforces.
- The escalation ladder is a computed label, not an active pager** — `escalation_rung` is computed as 0 (within SLA) or `min(int(age // sla_hours), len(ladder)-1)` once breached — each additional full SLA window climbs one rung (Team POC → Kaizen → GM → Finance/CXO), capped at the last rung. This produces a label the inbox surfaces; nothing in the code actually notifies or pages the Kaizen/GM/CXO person named by the rung. The ladder is informational escalation status, not automated routing.
- Resolving appends a linked follow-up; it never mutates the original** — `resolve(concern_id, note, resolver)` runs under a module-level `threading.Lock` so the check-then-append is atomic (two rapid clicks cannot double-resolve). It finds the original concern; if a follow-up already points to it, it returns `already_resolved` as a no-op (idempotent). Otherwise it appends a brand-new concern with `outcome="l3_resolved"`, `action_taken="resolved_by_l3"`, `resolves_concern_id=<original>`, the resolver name, the resolution note (or a default message), and it carries the original `evidence_trail` forward. The append-only design means resolution is a new event, so it simultaneously drops the case from the active inbox and becomes the captain-facing follow-up.
- The captain sees status via polling, not a push notification** — `cases(captain_id)` powers the captain-facing "My Cases" view. It builds a map of `resolves_concern_id → follow-up`, filters escalated

concerns to that captain, and returns each with `eta_hours` (from `TEAM_SLA`), status "resolved" if a follow-up exists else "open", plus `resolution_note` and `resolved_at`. The docstring states this is polled by the Captain Panel widget — so the resolve→notify loop is pull-based (the widget re-fetches and sees the new status); there is no push/WhatsApp message sent on resolve in this module.

- 7 **Each team owns its own metrics** — `team_metrics()` iterates the active inbox, groups by team, and counts open and breached per team (carrying `sla_hours`). This is the ownership view — each functional team sees its own queue and breach count. It is consumed by the ops dashboard: `/api/insights` returns `l3_teams` and an `active_breaches` count (breached items across the whole inbox).
- 8 **The governance scorer is an explicit stub** — `_governance_placeholder` is labelled a `PLACEHOLDER` in both the module docstring and its own docstring. It fakes a severity purely from amount (`>= ₹3000` = high, `>= ₹500` = medium, else low) and hardcodes recoverability and scalability to "unknown", returning `_placeholder: True` so downstream code can see it is not real. Every inbox item carries this stub under its governance key; the intended severity/recoverability/scalability model is TBD and swappable.
- 9 **Exposed through three authenticated API endpoints** — `main.py` wires `GET /api/l3/inbox` → `{items: inbox(), teams: team_metrics()}`, `POST /api/l3/resolve` (body: `concern_id`, `resolution_note`, `resolver`) → `resolve()`, and `GET /api/captain/{captain_id}/cases` → `{cases: cases(captain_id)}`. All sit behind the `_authenticated` dependency.

Key points

- The platform stores nothing of its own — the inbox, My Cases, and team metrics are all live views computed over the shared append-only Concern Log, so there is a single source of truth and no risk of the queue drifting from the ledger.
- Resolving is append-only and idempotent: the original concern is never edited, a linked follow-up event is added instead, guarded by a thread lock so double-clicks or concurrent resolves cannot create two resolutions.
- SLA hours are per-team (12/24/36/48h across 12 teams) and are the same numbers `engine/tools.py` quotes to the captain as an ETA, so the promise made and the deadline tracked are guaranteed to match.
- The escalation ladder (Team POC → Kaizen → GM → Finance/CXO) is displayed and advances with each missed SLA window, but nothing in the code actually pages or notifies those roles — it is a status label, not an automated escalation router.
- The captain 'resolve → notify' loop is polling-based: resolving creates a follow-up the My Cases widget picks up on its next fetch; no outbound push/WhatsApp message is sent from this module on resolve.
- The governance severity/recoverability/scalability scorer is a declared placeholder that guesses severity from the rupee amount alone and marks the other two dimensions 'unknown' — it must not be read as a real risk model.
- The `TEAM_SLA` table is described in a comment as externalized config (BRD §15.2) but is presently a hardcoded dict in the source; changing SLAs today means a code edit.

Status

BUILT AND WORKING: the accountable inbox as a live view over the Concern Log; per-team SLA lookup and breach detection; the ladder-rung computation; the append-only, thread-safe, idempotent resolve; the captain-facing 'My Cases' status view; per-team open/breached metrics; and the three authenticated API endpoints. All of this is real, functioning Python with no TODOs. EXPLICIT PLACEHOLDER/STUB: the `_governance_placeholder` severity scorer — flagged as a placeholder in the code itself, it derives 'severity' from

the rupee amount only and hardcodes recoverability and scalability to 'unknown' (a real scoring model is TBD, and the stub is wired only so the data shape is present and swappable). DESIGN-ONLY / NOT ACTUALLY WIRED: the escalation ladder climbs as a computed label but no code notifies or pages the Kaizen/GM/CXO roles it names; and the 'notify' side of resolve is pull-based (captain widget polling) rather than an outbound push. INFRASTRUCTURE CAVEATS: the underlying Concern Log is a JSON file for the demo (Postgres intended in production per the BRD), and the TEAM_SLA table, though commented as config, is hardcoded in the module today.

Key files

```
/Users/pranav.akella/PSP/backend/app/l3/platform.py ·  
/Users/pranav.akella/PSP/backend/app/ledger/concern_log.py ·  
/Users/pranav.akella/PSP/backend/app/engine/tools.py ·  
/Users/pranav.akella/PSP/backend/app/knowledge/policies.py ·  
/Users/pranav.akella/PSP/backend/app/main.py
```

9. Concern Log, Audit & Continuous Problem Discovery

Every interaction the platform handles is written once to an append-only Concern Log — a chronological, immutable record that is never edited in place; corrections and resolutions are added as new records, not overwrites. A companion Trace Log keeps a replayable, size-bounded copy of the step-by-step "how it worked the case" trace for each concern. On top of these sit two review mechanisms: a satisfaction loop that turns any "not satisfied" response into a Continuous Problem Discovery (CPD) signal for the support team, and an Auditing Studio where an LLM "judge" scores resolved concerns against an editable, versioned quality rubric and rolls the scores up into dashboard aggregates. All of this is built and wired to live API endpoints; the stores are JSON files mirrored to a hosted database (Turso) so they survive redeploys.

How it works

- 1 Every concern is appended once to the Concern Log** — `concern_log.append()` stamps each record with a UTC timestamp (`logged_at`) and a running sequence number (`seq = current length + 1`), appends it to the in-memory list, and rewrites the whole JSON file under a thread lock. Records are never updated after the fact — the log is append-only. Concerns are created from several places: the resolution engine when it applies a money policy (action `reverse_debit / clear_pendency`, or `escalate`), the engine's escalate-case tool, informational replies (disposition `'conversation'`, action `'respond'`), the proactive monitor (disposition `'proactive_nudge'`), and L3 when it resolves an escalated case.
- 2 L3 resolutions are appended as linked follow-up records, not edits** — `l3/platform.py resolve()` appends a NEW concern that carries `resolves_concern_id` (pointing at the original) and `action_taken='resolved_by_l3'`. This preserves append-only immutability: the original escalated record stays untouched, and the resolution is a separate, linked event. The call is guarded so a case can't be double-resolved.
- 3 Concern Log stats() computes headline counts, excluding shadow records** — It first filters to 'real' concerns (those WITHOUT `resolves_concern_id`) so the synthetic L3 follow-ups don't double-count resolved cases. From that set it reports: total; resolved_in_conversation (action in `reverse_debit / clear_pendency / respond` — note this also counts informational `'respond'` turns); escalated (action `escalate`); money_recovered_for_partners_inr (sum of `amount_inr` where action is `reverse_debit`); and by_disposition (a count per disposition).
- 4 resolution_time_stats() derives a best-effort mean resolution time straight from timestamps** — In-conversation resolutions are treated as instant (0 hours). For escalated concerns it looks up the linked L3 follow-up (via the `resolves_concern_id` map) and measures `elapsed = follow-up.logged_at - original.logged_at`, only when both timestamps parse and the follow-up is not earlier. If there are no measurable resolutions it returns display `'—'` rather than a fabricated number, and the whole function is wrapped so it can never raise and blank the insights panel. It returns the mean display, raw hours, sample size, in-conversation vs. via-L3 counts, and an L3-only mean.
- 5 The Trace Log keeps a bounded, replayable copy of each resolution's stage trace** — The engine streams a live trace (`capture -> tools -> checks -> decision -> reply`) that is otherwise ephemeral. `conversation.py's _persist_trace()` (run in a finally block, fully wrapped in try/except so a save failure never breaks the turn) calls `trace_log.save()`, keyed by `concern_id`. `save()` trims each event (strings capped ~500 chars, arrays ~10 items) to keep the store small and caps the whole store at 2000 traces, dropping the oldest by save time. A multi-intent turn attaches the same trace to each concern it produced. `get()` powers both the `/api/concern/{id}/trace` expander in the UI and the LLM judge.
- 6 The satisfaction loop records outcomes and mints CPD signals** — A client posts to `/api/satisfaction`, which calls `cpd.record_satisfaction()`. It appends an entry (`id CPD-xxxxxx`) with `satisfied true/false`; a dissatisfied response sets `is_cpd=true` and `status='open'`, a satisfied one is

'closed'. `satisfaction_stats()` reports responses, satisfied count, `csat_pct` ($100 \times \text{satisfied} / \text{total}$, or null when there are no responses), and `open_cpd` count. `cpd_items()` returns only the CPD (dissatisfied) items, newest first.

- 7 **`audit_trail()` projects a fair per-concern chain for review** — For each of the most recent concerns (default 50) it reads the immutable log and flattens the chain a reviewer wants: `who` (`captain_id`, `channel`), `disposition/intent`, `data_used` (the evidence-trail sources), `action_taken`, `confidence`, `outcome`, `turns`, `reply`, and `logged_at`. It is a read-only view over the Concern Log.
- 8 **The Auditing Studio rubric is editable, versioned data (not code)** — `rubric.py` seeds version 1 with six weighted dimensions summing to 1.0: Identification 0.20, Grounding 0.15, Decision 0.20, Partner-supportedness 0.25, Efficiency 0.10, Safety 0.10. `save_rubric()` normalises author input (deriving a key from the label, coercing weights to non-negative floats), requires at least one dimension, and increments the version on every save (stamping `updated_at`). Weights need not sum to 1 — the judge normalises at scoring time. Saving is an authoring-privileged write (POST `/api/audit/rubric`).
- 9 **The LLM judge scores one concern against the current rubric** — `audit_concern()` assembles the full picture a human QA reviewer would see — the captain's concern fields, the rendered resolution trace from the Trace Log, and the final reply — and asks the deep-tier LLM (same registry node as the SOP compiler, `sop_compile`, in JSON mode) to score each rubric dimension 0.0–1.0 with a one-line rationale plus an overall verdict. It retries once on a JSON parse miss and falls back to zeros on failure. Scores are coerced so every rubric key is present and clamped to [0,1]. The composite is a weighted 0–100 score normalised by the sum of weights. The result is persisted to `audits.json` stamped with `rubric_version` and `audited_at`.
- 10 **`audit_batch()` samples the most recent un-audited concerns; it is on-demand** — It collects the newest concerns not already in `audits.json`, up to the requested limit (default 10), audits each, and returns audited count, `avg_composite`, the individual results, and how many were already audited. By design it does NOT audit everything. It only runs when POST `/api/audit/run_batch` is called — there is no scheduler/cron auto-running it.
- 11 **`scores()` aggregates the audit history for the dashboard** — From `audits.json` it computes: count, overall `avg_composite`, `per_dimension_avg` (mean score per dimension across all audits), a by-day `time_series` (`avg_composite` + count), a by-disposition breakdown (`avg_composite` + count), the current `rubric_version` and dimensions, and the full history newest-first. It returns a safe empty shape when no audits exist.

Key points

- The Concern Log is genuinely append-only: `append()` only ever adds rows and rewrites the file; nothing updates a record in place. Resolutions/corrections are new linked records (`resolves_concern_id`), which is why `stats()` must filter out those shadow follow-ups to avoid double-counting.
- Stats are computed on demand from the raw log, not stored counters. `'resolved_in_conversation'` includes informational `'respond'` turns as well as money resolutions, and `'money_recovered_for_partners_inr'` counts only reverse_debit amounts.
- `resolution_time_stats()` is honest by construction: unmeasurable cases are excluded and, when nothing is measurable, it shows `'—'` instead of a made-up figure; it can never raise.
- The Trace Log is a best-effort sidecar: it trims event data (~500-char strings, ~10-item arrays) and hard-caps at 2000 traces, and its save is wrapped so a failure never affects the live conversation. It is what makes any resolution replayable and is what the LLM judge reads.

- The 'satisfaction loop' in the backend is a recording endpoint, not an automated survey: the client decides when to ask the partner and posts the result. The backend just logs it and flags dissatisfaction as a CPD item.
- CPD items are logged-only. A dissatisfied response creates an item with status 'open', but no code in these files ever closes, actions, or auto-converts a CPD item into an SOP/KT/data-fix — that downstream action is manual/human follow-up (the docstring's 'new SOP / KT / data gap' is the intent, not automation).
- Auditing is sampling and on-demand: the batch judges only the most recent N un-audited concerns and runs only when an operator hits the endpoint. There is no scheduled or continuous auditing.
- The rubric is editable, versioned data; every audit is stamped with the rubric_version it was scored under so scores stay comparable within a version. The composite normalises by the sum of weights, so weights don't have to add to 1.
- The LLM judge is a scorer, not a resolver — the prompt explicitly forbids it from re-resolving the case. Its output is defensively coerced and clamped, and on repeated LLM/JSON failure the concern still gets a (zero-score) audit record rather than an error.

Status

Production-ready and working end to end: the append-only Concern Log (append + newest-first read), on-demand stats() and resolution_time_stats(), the bounded Trace Log sidecar, CSV/JSON export, the satisfaction-recording endpoint with CSAT/CPD stats, audit_trail(), the versioned editable rubric, the LLM judge (audit_concern / audit_batch), and the scores() dashboard aggregates — all reachable via /api endpoints and persisted to JSON files mirrored into Turso (libSQL) so they survive redeployes; with Turso unconfigured they degrade cleanly to plain local files. Honest caveats: (1) the stores are simple whole-file JSON rewrites over an in-process list/dict — fine for a demo, but not a scalable event store; the code's own comments note Postgres 'in production' is aspirational/design-only, not implemented here. (2) CPD is discovery/logging only — dissatisfaction is captured and surfaced (open_cpd, cpd_items), but nothing automatically closes or actions those items or turns them into SOPs/KT; that is a manual/human step. (3) Auditing is manual and sampled — no scheduler runs the batch, and by default only the most recent un-audited concerns are judged, so 'continuous' auditing is operator-triggered, not automatic. (4) Judge quality depends on an external LLM; on failure it still writes an audit record but with zeroed scores, which would drag composite/aggregates down rather than surface as an explicit error.

Key files

backend/app/ledger/concern_log.py · backend/app/ledger/trace_log.py ·
 backend/app/audit/cpd.py · backend/app/audit/rubric.py · backend/app/audit/runner.py ·
 backend/app/durable_state.py · backend/app/engine/conversation.py ·
 backend/app/engine/tools.py · backend/app/monitor/monitor.py · backend/app/l3/platform.py ·
 backend/app/main.py

10. Data Substrate & Durability

The platform reads facts from two independent grounding paths and never lets the model invent them. (1) Captain Context assembles one auditable per-captain view (profile, payments ledger, losses, cash/COD, plus Log10 shipment scans), and today it is served entirely from canned seed data for three demo captains — no live database is connected. (2) The Loss DB answers per-AWB dispute lookups against a real, PII-stripped subset of the team's Metabase loss export, loaded into SQLite (or the same data pushed to a hosted Turso database) — this is genuine operational data, not synthetic. Separately, because Render's free tier has no persistent disk, all authored/operational state (users, SOPs, governance framework, audit rubric, logs) is mirrored to Turso so it survives redeployments, with explicit safeguards against a transient outage wiping durable content.

How it works

- Captain Context composes two mock providers** — `captain_context.py` builds `get_context()` by combining `DemoDataProvider` (Meesho account data: profile/ledger/losses/cash) and `Log10Connector` (shipments/scans), tagging each field with its source in `_sources` so provenance is auditable. Both providers return canned rows from `seed.py` today. `mock_connectors.py`'s own docstring calls `DemoDataProvider` 'the LIVE data provider today (canned Metabase query results)'. No live account DB is wired in.
- Seed data covers three scripted demo captains** — `seed.py` hardcodes VLMO-CPT-4471 (Rajesh, clean account + a shipment nearing a D5 breach for the monitor demo), VLMO-CPT-2290 (Anita, COD pendency), and VLMO-CPT-3310 (Vikram, the wrong-debit-reversal hero with a full scan trail). These, and only these, are what the conversation/monitor see for captain context.
- Live account/shipment integrations are designed but stubbed** — `adapters/experimental/metabase_provider.py` is a stub: it defines a named-query registry (`get_loss_attribution`, `get_cod_pendency`, etc.) but `query()` raises `NotImplementedError` and the README says it is 'Not wired into the runtime.' `Log10Connector` similarly raises `NotImplementedError` inside `_live()` if `LOG10_API_BASE/LOG10_API_KEY` are set. The `DataProvider Protocol` in `experimental/base.py` fixes the contract so swapping mock→live is an env/provider change, not an engine change.
- Loss DB grounds real dispute resolution per-AWB** — `loss_db.py._init()` picks a source once: if `TURSO_DATABASE_URL + TURSO_AUTH_TOKEN` are set it probes remote Turso with 'SELECT 1' and uses mode 'remote'; otherwise it opens the local file read-only — `valmo.db` (514MB full) if present, else `valmo_fallback.db` (15MB subset baked into the image); otherwise 'none'. The same SQL runs on both because `libSQL` is SQLite-compatible. Any remote failure silently falls back to local, so the demo can't break.
- Loss lookups consolidate multiple tables** — `get_loss_by_awb()` tries tables in priority order — losses (full export) → `loss_attrib` → attribution ledger → `qc_fail` — normalizing each into one loss-row shape and collapsing multi-row AWBs (a sign attribution changed). `policy_exec.py` calls this plus `get_pendency/get_attrib_change/get_attribution` to feed the resolution engine real signals (`facility_inscan`, `attribution_changed`, `loss_percentage`). If the AWB isn't found (or `loss_db.available()` is false), the engine files an honest 'couldn't locate / no connectivity' escalation rather than dead-ending.
- Loss data originates from a real Metabase export** — `build_valmo_db.py` ingests the team's Metabase loss-query CSV (`gold.valmo_lost_awb...`), drops four PII columns, keeps AWB/amounts/reason/reversal signals, and loads a stratified subset into SQLite indexed on AWB — plus enrichment tables (`attribution`, `pendency`, `attrib_change`, `loss_attrib`, `qc_fail`) from companion exports. It injects zero synthetic rows unless `--seed-demo/SEED_DEMO=1` is passed, which adds one curated hero AWB (VL0093310077) so the scripted reversal demo runs through the real data path.

- 7 **push_to_turso.py replicates SQLite into hosted Turso** — A pure-Python script (no Turso CLI) reads local valmo.db and recreates each table in Turso as all-TEXT columns, bulk-inserting in 1000-row batches via `turso_http.batch()` and creating an awb index. Once pushed, the deployed image can ship without the heavy data file and the engine queries Turso instead.
- 8 **turso_http.py is the minimal HTTP client** — It talks to Turso/libSQL's /v2/pipeline HTTP API using requests + certifi (no WebSocket/native deps, so it runs identically on macOS and the Linux container). Everything is stored/read as TEXT (Hrana encoding). `execute()` runs one statement; `batch()` wraps many statements in a single BEGIN/COMMIT transaction in one round-trip (noted as ~100x faster than per-row autocommit). One pooled HTTPS session is reused across queries.
- 9 **durable_state.py mirrors mutable JSON stores to Turso** — Because `$PSP_STATE_DIR (/data on Render)` is wiped on every deploy/idle restart, each state file is mirrored into a Turso key-value table `psp_state (k, v, updated_at)`. `durable_path(name)` returns a Path-like handle whose `exists/read_text/write_text` go through Turso; the local file is kept only as a fast per-process cache while Turso is the durable truth. `write-through _put()` does `INSERT ... ON CONFLICT DO UPDATE` and updates an in-process cache. If `TURSO_*` is unset (local dev), it transparently degrades to plain local files with identical behavior.
- 10 **read_confirmed provides clobber-safety** — `read_confirmed(name)` returns (ok, value). `ok=False` means a durable read was ATTEMPTED but failed (transport error) — the caller must NOT rewrite the store, because on the free tier the local cache may be wiped and a reconcile-then-save would overwrite durable truth with seeds-only. `_fetch()` caches successful reads but deliberately never caches transport errors, so a blip is never mistaken for 'genuinely absent'. `kt/engine.py` and `sop_compiler.py` honor this: on `ok=False` they serve baked seeds in-memory and write nothing until Turso recovers.
- 11 **Baked-in seeds re-hydrate after a wipe** — Authored stores seed themselves on first run from code-baked defaults: the KT queue/SOP seeds (`_seed_policies`), the Losses blueprint (`_seed_if_empty`), the governance framework placeholder (`_seed`), the audit rubric v1 (`_seed`), and `INITIAL_ADMIN` from env. Seeds survive an ephemeral reset because they live in code; user edits always win and are never duplicated (an edited seed drops its 'seeded' flag and is left untouched).

Key points

- Two separate data paths with different fidelity: Captain Context is 100% mock/canned (three demo captains from `seed.py`); the Loss DB used for dispute resolution is a real, PII-stripped Metabase export subset queried per-AWB.
- The only synthetic loss record is the demo hero AWB VL0093310077, injected only under `--seed-demo/SEED_DEMO=1`; production builds inject zero synthetic rows.
- Live account-data (Metabase) and shipment (Log10) integrations are designed contracts only — `MetabaseProvider.query()` and the Log10 live branch both raise `NotImplementedError`, and the Metabase provider README says it is not wired into the runtime.
- Turso serves double duty: it hosts the loss data (optional, when `TURSO_*` env is set) and it is the durability layer for all mutable authored/operational state via the `psp_state` KV table.
- Every remote/Turso path is failure-guarded: `loss_db` falls back `remote → local valmo.db → baked valmo_fallback.db → none`; `durable_state` falls back to local files if Turso is unconfigured or errors.
- `read_confirmed` is the key durability safeguard: it distinguishes 'genuinely absent' from 'couldn't reach Turso' so a transient outage on the disk-less free tier can never let a re-seed clobber authored SOPs/content.
- `turso_http` uses plain HTTPS (requests+certifi, TEXT-only values) rather than native libSQL/WebSocket drivers, keeping it dependency-light and identical across macOS dev and the Linux container.

- The Turso database URL (libsql://psp-panna-505.aws-ap-south-1.turso.io) is committed in backend/data/turso_url.txt; the auth token and other secrets are Render dashboard secrets (sync: false, never in git).

Status

BUILT AND WORKING: (1) Captain Context serving canned seed data for three demo captains — this is the live provider today, fully functional but mock. (2) Loss DB per-AWB lookup over real PII-stripped Metabase export data in local SQLite, wired into the resolution engine; enrichment-table consolidation works. (3) Turso HTTP client (execute/batch). (4) push_to_turso.py replication and build_valmo_db.py ingest scripts. (5) durable_state mirroring, write-through cache, read_confirmed clobber-safety, and baked-seed re-hydration — all implemented and used by the KT/blueprint/governance/rubric/auth stores. Whether Turso is actually the live source at runtime depends entirely on TURSO_DATABASE_URL + TURSO_AUTH_TOKEN being set (Render dashboard secrets); with them unset everything degrades to local files, which is the default local-dev behavior.

DESIGN-ONLY / NOT WIRED: the live Metabase account-data provider (metabase_provider.py is a stub raising NotImplementedError, README says not wired) and the Log10 live API calls (raise NotImplementedError when LOG10_ env is set). Both have finalized contracts/scaffolding but no working live implementation. The synthetic hero AWB is demo-only and explicitly excluded from production builds.*

Key files

backend/app/substrate/captain_context.py · backend/app/substrate/loss_db.py ·
 backend/app/substrate/seed.py · backend/app/substrate/turso_http.py ·
 backend/app/substrate/adapters/mock_connectors.py ·
 backend/app/substrate/adapters/log10_connector.py ·
 backend/app/substrate/adapters/experimental/metabase_provider.py ·
 backend/app/substrate/adapters/experimental/base.py ·
 backend/app/substrate/adapters/experimental/README.md · backend/app/durable_state.py ·
 backend/app/state_paths.py · backend/app/engine/policy_exec.py · backend/app/kt/engine.py ·
 backend/scripts/build_valmo_db.py · (+3 more)

11. Proactive Monitoring

Proactive Monitoring reuses the same resolution engine but enters from a data-change event instead of a partner chat. It runs a cheap deterministic rule-pass over a captain's shipments and only invokes an LLM when a genuine risk fires — so LLM cost tracks the number of risk-events, not the number of captains. When a risk fires it composes a short, partner-protective "nudge" and writes it to the shared Concern Log, so proactive protection appears in the same Ledger/Audit/Insights views as reactive resolutions. In the current build every nudge is flagged shadow-first and logged only; there is no captain-facing delivery, no background worker, and the underlying shipment "stream" is canned seed data. The "always-on, all-captains" behavior is the design intent, implemented today as an on-demand, one-captain-at-a-time scan.

How it works

- 1 Entry point is an on-demand HTTP scan, not a background consumer** — GET `/api/monitor/{captain_id}` (backend/app/main.py:252, auth-guarded) calls `monitor.scan_captain(captain_id)` and streams its trace as Server-Sent Events. The frontend triggers it per captain via a 'Run scan' button. There is no scheduler, queue consumer, cron, or background worker in the codebase — the 'always-on headless consumer of the Log10/Captain-Panel event stream' described in the UI banner is design framing, not built.
- 2 Load the captain's shipments (canned seed today)** — `scan_captain` calls `captain_context.get_context`, whose shipments come from `Log10Connector.get_shipments`. That connector returns seed rows unless `LOG10_API_BASE + LOG10_API_KEY` are set — and if they are set the live path deliberately raises `NotImplementedError` ('Wire ... at go-live'). So the 'event stream' is static seed data; the default captain VLMO-CPT-4471 carries two shipments, one already primed to trip the rule.
- 3 Cheap deterministic first-pass — no LLM** — `_cheap_first_pass` runs pure Python per shipment: it looks up the SLA window for the shipment's (leg, direction) from the SLA matrix in `substrate/seed.py`, then fires a 'hardstop_breach_imminent' risk when the shipment is NOT on the correct manifest path AND `days_since_inscan >= (within - 1)`, i.e. one day before the SLA 'within' window closes. For LM/Forward that is `within=5`, so a 4-day-old un-manifested shipment fires; the risk carries the hardstop day (D7 for LM/Forward per the matrix). This is the only rule implemented.
- 4 Skip the LLM entirely when nothing is at risk** — The scan emits a 'firstpass' trace event with counts, then if the risk list is empty it yields a 'clear' event ('Nothing to warn about — cost stayed at ~zero (no LLM)') and returns. This is the mechanism behind the cost-discipline claim: the LLM is reached only when a risk has already fired deterministically.
- 5 Compose one nudge per fired risk via the model registry** — For each risk, `_compose_nudge` asks `llm_registry.for_node('monitor_compose')` for the provider/model. That node is tier 'fast'; with the active config (provider: openai in `config/models.yaml`) it resolves to gpt-5.5 via the Bifrost gateway. The prompt asks for a 1-2 sentence warm, partner-protective warning in the captain's language, telling them to fix the manifest path before the hardstop breach 'so they are not wrongly debited.'
- 6 Write every nudge to the shared Concern Log** — `concern_log.append` records the nudge with a new CNC- id, `channel='proactive'`, `disposition='proactive_nudge'`, `action_taken='nudge_sent'`, `outcome='proactive_nudge'`, and an `evidence_trail` citing the risk (source 'monitor.first_pass'). Because outcome is 'proactive_nudge' and NOT 'escalated', these records never enter the L3 human-review inbox, but they do surface in the Ledger/Audit/Insights that read the same append-only log (`concern_log.json`).

- 7 **Emit the nudge event flagged shadow-first** — The final trace event carries data with shadow: True (hard-coded), the composed nudge text, the model used, and the concern id. Nothing sends the nudge to the captain — there is no SMS/WhatsApp/push path in the code. In practice the build is 'shadow-only': the nudge is composed and logged, and no mechanism exists yet to graduate a nudge out of shadow into a live partner alert.
- 8 **Frontend renders the trace live** — frontend/src/pages/Monitor.jsx consumes the SSE stream, appends each event, and renders it through the shared Pipeline component. A DecisionCore 'orb' pulses on each event and decays every animation frame — purely decorative cadence, not a real load signal. A captain dropdown (default VLMO-CPT-4471) selects whose shipments to scan; explanatory copy states that at scale it 'runs continuously across all captains on Log10.'

Key points

- Cost model is real in the code: the LLM is invoked strictly per fired risk, and the no-risk path returns before any LLM call. But the 'scales with risk-events, not captain count' benefit only fully materializes under the aspirational always-on design; the implemented endpoint scans ONE captain per HTTP call, triggered by a button.
- Only one deterministic rule exists: 'hardstop_breach_imminent' for un-manifested shipments approaching the SLA window. There is no other risk detector.
- The trace label reads 'Cheap first-pass (rules + vector)', but the code does NO vector/embedding search — the first-pass is rules-only. 'vector' is a label string only (confirmed: the sole 'vector' occurrence in the module is that display string).
- The code label 'Compose nudge (Haiku)' is aspirational — Haiku (claude-haiku-4-5) is only used if config/models.yaml is flipped to provider: claude. With the active provider (openai) the nudge is composed by gpt-5.5.
- shadow: True is hard-coded on every nudge and there is no captain-facing send path, so the build is effectively shadow-only; there is no implemented switch to turn a nudge into a real partner alert.
- Monitoring genuinely reuses the shared spine: nudges land in the same append-only Concern Log as reactive resolutions, tagged so they never reach the L3 human inbox (outcome != 'escalated').
- The shipment data is canned seed; the live Log10 path is an explicit NotImplementedError stub, so there is no live event stream today.
- SLA windows are seeded in code (substrate/seed.py): LM/Forward is within=5, hardstop=7 — note the demo shipment's human 'note' text says 'D5' but the logic-driven hardstop passed to the nudge is D7 for that leg.

Status

BUILT AND WORKING: the SSE endpoint; the deterministic hardstop-breach rule; the skip-LLM-when-no-risk path; per-risk LLM nudge composition through the model registry (active model gpt-5.5); appending each nudge to the append-only Concern Log tagged so it never enters L3; and the Monitor.jsx UI (captain selector, live trace, decorative orb). DESIGN/ASPIRATION, NOT BUILT: (1) 'Always-on headless consumer of a live event stream running continuously across all captains' — implemented as an on-demand, single-captain HTTP scan with no scheduler/worker/queue; (2) live Log10 shipment data — canned seed only, live path raises NotImplementedError; (3) the 'rules + vector' first-pass — vector search does not exist, rules only; (4) the 'Haiku' compose label — only active if the provider is flipped to Claude; today it is gpt-5.5; (5) shadow-first graduation — shadow is hard-coded True and there is NO captain-facing delivery mechanism, so nudges are composed and logged but never actually sent. Net: the detection-to-log pipeline is real and demonstrable on seed data; the 'proactive protection reaching the partner at scale' story is not yet implemented.

Key files

backend/app/monitor/monitor.py · frontend/src/pages/Monitor.jsx · backend/app/main.py ·
backend/app/substrate/captain_context.py · backend/app/substrate/adapters/log10_connector.py
· backend/app/substrate/seed.py · backend/app/ledger/concern_log.py ·
backend/app/llm/registry.py · backend/config/models.yaml

12. Authentication, Roles & Security

The platform is access-controlled from the first request: every API route except two (login and the health check) requires a valid signed session token, and sensitive actions additionally require the right role. Three roles exist in the backend — author, approver, viewer — which the UI relabels as Editor, Admin, and Viewer respectively (a display-only relabel; the stored values never change). Passwords are salted and hashed with PBKDF2, sessions are compact HMAC-signed tokens with a 12-hour expiry, and the caller's role is read only from the signed token — the client is never trusted to assert its own role. Secrets (auth signing key, LLM keys, database tokens) are injected as environment variables and kept out of the repo. The live connection to the organization's data lake (Presto/Trino behind Metabase) is designed but deliberately not built: the production data provider is a stub, and the app instead runs on canned demo data plus a baked, PII-stripped snapshot.

How it works

- 1 An admin (approver) seeds or creates accounts** — On startup (`main.py @app.on_event("startup")` → `auth_store.seed_initial()`), if the user store is empty exactly ONE approver is created — from `INITIAL_ADMIN_EMAIL` / `INITIAL_ADMIN_PASSWORD` env vars if set, otherwise a documented default `admin@valmo.local` / `valmo-admin`, printing a `stderr WARNING` to change it before the pilot. Thereafter approvers add teammates via `POST /api/auth/users` (email, name, role, temp password). There is no self-service signup — the login screen tells users to 'Ask an admin on your team to create your account.'
- 2 Passwords are stored hashed, never in plaintext** — `store.py _make_user` generates a per-user random 16-byte salt (hex) and stores `pw_hash = pbkdf2_hmac('sha256', password, salt, 200_000)`. `users.json` holds {email, name, role, salt, pw_hash}. The record with salt/hash is only returned by `get_user()` for internal auth checks; every API-facing read (`list_users`, `verify_password`'s return, the seed) passes through `_public()` which strips salt and pw_hash.
- 3 User logs in and receives a signed token** — `POST /api/auth/login` is public. `auth_store.verify_password()` recomputes the PBKDF2 hash and compares it to the stored hash using `hmac.compare_digest` (constant-time); on match it returns the public user view, and the endpoint issues a token via `tokens.make_token(email, role)`. Bad credentials → HTTP 401 'invalid email or password'.
- 4 The token is a self-contained HMAC-signed string** — `tokens.py`: `token = b64url(json({email, role, exp})) + '.' + b64url(HMAC-SHA256(SECRET, body))`, where `exp = now + 12h`. It is stateless (no server-side session table). The signing SECRET comes from env `AUTH_SECRET`; if unset the code falls back to a fixed dev string ('`psp-dev-auth-secret-change-me`') and warns ONCE on `stderr` — production MUST set `AUTH_SECRET` or tokens could be forged.
- 5 Every subsequent request carries the token; the server verifies it** — The frontend stores the token in `localStorage` (key '`valmo.token`') and sends it as '`Authorization: Bearer <token>`'. `deps.current_user` extracts it (Bearer header or X-Auth-Token fallback) and calls `tokens.verify()`, which recomputes the signature (constant-time compare) and checks exp is in the future; a missing token → 401 'authentication required', a bad/expired token → 401 'invalid or expired token'. The {email, role} returned comes entirely from the verified token payload.
- 6 Role gates enforce who can do what, server-side** — `main.py` defines three dependency gates: `_authed` (any authenticated user), `_author = require_role('author')`, `_approver = require_role('approver')`. `require_role` (`deps.py`) 403s ('requires role: ...') unless the caller's role is allowed, with one rule: an approver implicitly satisfies any author-level gate (`approver ≥ author`). Reads use `_authed`; authoring writes (`sop/blueprint compile+save`, `kt/submit`, `sop/nuance`, `audit/rubric save`,

framework save+upload) require `_author`; approvals, go-live, deletes and team management (sop/approve, sop/delete, blueprint/approve, kt/review, framework/approve, GET+POST `/api/auth/users`) require `_approver`.

- 7 **The UI mirrors the same rules but is not the enforcer** — `auth.jsx` computes `isApprover` / `isAuthor` (`approver ≥ author`) purely to decide which screens and buttons to show; `App.jsx` maps the backend roles to display labels `approver→Admin`, `author→Editor`, `viewer→Viewer`. A 401 from any call clears the stored token and fires a 'valmo-unauthed' event that returns the app to the login screen. Logout is client-side only (clears `localStorage`); because tokens are stateless there is no server-side revocation — an issued token stays valid until its 12h expiry.
- 8 **Accounts survive deploys via durable state** — `store.py` reads/writes `users.json` through `durable_state.read_json/write_json`, which mirrors the file into Turso (libSQL) so accounts persist across Render free-tier restarts, falling back to the plain local file when Turso is not configured.

Key points

- Three backend roles — `author`, `approver`, `viewer` — relabeled in the UI as Editor / Admin / Viewer (display only; stored values are unchanged). `approver ≥ author`, so an approver passes any author-level gate.
- Role is authoritative from the signed token only; the client cannot assert its own role. Enforcement is server-side on every gated route via FastAPI dependencies (`require_role → 403`).
- Exactly two routes are public: `POST /api/auth/login` and `GET /api/health`. All other `/api` routes require a valid token.
- Passwords: PBKDF2-HMAC-SHA256, 200,000 iterations, per-user 16-byte random salt; verified with constant-time compare. Salt and hash are never returned by any API — public views strip them.
- Sessions: stdlib-only HMAC-SHA256-signed tokens (no external JWT lib), 12h expiry, stateless. Signature verified with constant-time compare; malformed tokens are caught and treated as 401, never a 500.
- `AUTH_SECRET` must be set in production; an unset secret triggers an insecure documented dev fallback with a one-time stderr warning (forgeable tokens if left unset).
- Initial-admin seeding creates one approver on first boot from `INITIAL_ADMIN_*` env or a documented default (`admin@valmo.local / valmo-admin`) with a change-me warning.
- Team management is approver-only and minimal: list users and create users (email, name, role, temp password). There is no update, delete, password-reset, or self-service-signup endpoint. `CreateUserIn` defaults role to 'viewer'.
- Secrets are env-injected and gitignored: `AUTH_SECRET`, LLM keys, Turso URL/token, `INITIAL_ADMIN_*`. `.gitignore` excludes `backend/data/*.txt`, `.env`, `*.db` and `*.json` — local secret files (`llm_key.txt`, `turso_token.txt`, `turso_url.txt`) exist on the dev machine but are not committed.
- PII handling is partial-by-design: the shipped loss snapshot is PII-stripped (`build_valmo_db.py` DROPS `transporter_or_FE_name`, `accepted_by`, `requested_by`, `remarks`), and `metabase_probe.py` redacts secrets in diagnostics. Broader controls (masking identifiers in traces/logs, retention limits) are stated in `ROLLOUT.md` as intended-before-real-data, not yet implemented.
- The live production data-layer connection is designed but deliberately NOT built: `MetabaseProvider` is an experimental stub that raises `NotImplementedError`, and the runtime instead uses canned demo data plus a baked ~37k-row PII-stripped SQLite snapshot. The `Metabase→app-DB` sync is a backlog item blocked on data-team access.

- CORS is fully open (allow_origins=['*'], all methods/headers) — the token gate, not CORS, is the access control; this is a posture gap to tighten for production.

Status

BUILT AND WORKING: login with PBKDF2-hashed passwords; 12h HMAC-signed stateless tokens (Bearer/X-Auth-Token); server-side role enforcement on every gated route (author/approver/viewer with approver ≥ author); the two public routes (login, health); initial-admin seeding; approver-only list-users and create-user; UI role relabel (Editor/Admin/Viewer) and login-gated app shell; durable users.json via Turso with local fallback. All of this is real, exercised code — not stubs.

DELIBERATE DESIGN CHOICES (not defects): tokens are stateless, so logout is client-side only and there is no early revocation before the 12h expiry; the dev AUTH_SECRET fallback and default admin@valmo.local seed are documented conveniences that require overriding in production (both self-warn).

DESIGNED / INTENDED BUT NOT BUILT: the live data-lake connection — MetabaseProvider.query() raises NotImplementedError and the app runs on demo data plus a baked snapshot; the Metabase→app-DB sync is backlogged, blocked on data-team access. Deeper PII controls (masking identifiers in traces/logs, retention limits) are described in ROLLOUT.md as landing with the real-data sync, not yet implemented. Team management has no edit/delete/password-reset. CORS is wide open. NOTE ON THE '55TB' FIGURE: I could not find any literal '55TB' / terabyte size in the code or docs — the source is documented only as an org Presto/Trino data lake behind Metabase (docs/BACKLOG.md A1). The 'designed but deliberately not built' characterization of that connection is fully accurate to the code; the specific 55TB size is not something the repo states, so I have not asserted it as a code fact.

Key files

backend/app/auth/store.py · backend/app/auth/tokens.py · backend/app/auth/deps.py ·
 backend/app/auth/__init__.py · backend/app/main.py · backend/app/durable_state.py ·
 backend/app/state_paths.py ·
 backend/app/substrate/adapters/experimental/metabase_provider.py ·
 backend/scripts/build_valmo_db.py · backend/scripts/metabase_probe.py ·
 frontend/src/lib/auth.jsx · frontend/src/lib/api.js · frontend/src/App.jsx · docs/ROLLOUT.md
 · (+1 more)

13. Channels & Voice / Conversation Mode

The platform is built around one resolution engine with interchangeable "transports": today the working transport is the in-app Captain Panel (web chat), and a WhatsApp webhook adapter is wired to the same engine but drives it locally — it parses inbound messages and generates real replies, but does not actually send anything back over WhatsApp yet. On top of the web chat, the Captain Panel adds two voice features: single-shot dictation and a hands-free "Conversation mode" (listen → answer → speak → listen), both powered entirely by the browser's built-in Web Speech API. A reactive 3D "Decision Core" orb visualizes the engine's state and, in voice mode, physically reacts to the captain's live microphone volume. Server-side speech (Whisper/Sarvam) is labelled and designed for but is not built — the "Sarvam-ready" text is a swap-point marker, not a working integration.

How it works

- 1 One engine, two channels (design premise)** — `channels/whatsapp.py` states the intent plainly: WhatsApp and the Captain Panel are two versions of the same engine — different transports, one brain. The WhatsApp file is a thin adapter that maps a webhook payload to a conversation turn and formats the reply; the engine itself is unchanged. The engine turn is even told which channel it came from via `handle_turn(..., channel="whatsapp")`.
- 2 WhatsApp inbound parsing (works)** — `whatsapp.parse_webhook(payload)` accepts TWO shapes: a simplified `{from, text, media}` shape for the demo/simulator, and the real Meta Cloud API nested shape `entry[].changes[].value.messages[]`. It pulls out phone, text, and any media (image/audio/document ids), and resolves the sender to a captain via `PHONE_TO_CAPTAIN` — a hard-coded 2-entry demo lookup (919000000001 → VLMO-CPT-4471, etc.). Returns `None` if there is no user message.
- 3 WhatsApp webhook endpoint (drives the engine locally)** — `main.py` exposes `POST /api/whatsapp/webhook` (behind the app's auth dependency). It builds a payload from `body.payload` or `{from, text}`, parses it, rejects unknown sender / empty message, keys the conversation by phone (`conv_id = "wa-<phone>"`, so multi-turn works over WhatsApp), runs `conversation.handle_turn(...)`, captures the engine's terminal 'reply' node, and returns `{captain_id, state, reply, outbound}`. So a posted webhook produces a genuine engine-generated reply.
- 4 WhatsApp outbound send (STUB — never actually sends)** — `whatsapp.send(phone, text)` is a placeholder. If `WHATSAPP_TOKEN` and `WHATSAPP_PHONE_ID` env vars are present it deliberately raises `NotImplementedError` ('Wire POST `https://graph.facebook.com/v20.0/{phone_id}/messages` here at go-live'); with no creds (the demo default) it just returns a demo-echo of the outbound payload. Nothing is transmitted to WhatsApp. Going live = provisioning a Meta Business account + verified number + permanent token + public HTTPS webhook and replacing `send()` with a Graph API POST (per `PRODUCTION_DELTA`).
- 5 In-app dictation (single-shot STT, works in Chrome/Edge)** — `CaptainPanel.toggleMic()` uses the browser Web Speech API (`window.SpeechRecognition` / `webkitSpeechRecognition`). It runs one recognition pass (`interimResults` on, `continuous` off), streams the transcript into the chat composer textarea, then stops. The captain still reviews and presses send. If the API is unavailable it alerts 'Voice input needs Chrome/Edge'.

- 6 **Hands-free Conversation mode loop (works, browser-only)** — `startConvo()` opens a full-screen overlay and starts a half-duplex loop: `listen()` (SpeechRecognition) → on a final transcript calls `send(text)` → the `/api/chat` SSE stream returns a reply → `speak(replyText, () => listen())` uses `window.speechSynthesis` (SpeechSynthesisUtterance) to read it aloud, and only when the utterance ends does it listen again. Mic is off while speaking so TTS doesn't feed back into the mic. If nothing is heard it retries `listen()` after 350ms. `convoRef/convoIdRef` mirror state into refs because the loop's closures would otherwise freeze React state and re-use of one `conversation_id` keeps the hands-free session as a single thread.
- 7 **Language handling (8 Indic languages)** — `VOICE_LANGS` lists 8 BCP-47 codes (hi, en, mr, ta, te, kn, ml, bn-IN), persisted to `localStorage` as `valmo.voiceLang`. The chosen code is applied to both STT (`rec.lang`) and TTS (`utterance.lang`); `_pickVoice()` tries an exact browser-voice lang match, then a 2-letter prefix, else null (browser default). Whether a real Indic voice is available depends entirely on the viewer's browser/OS installed voices.
- 8 **Interrupt & end** — Tapping the orb while the bot is speaking (`interruptBot`) cancels `speechSynthesis` and jumps back to `listen()`; taps during 'thinking' are ignored. An 'End conversation' button calls `stopConvo()` which clears the loop flag, cancels speech, stops the mic meter, and closes the overlay. Cleanup also runs on `unmount`.
- 9 **Live mic-amplitude meter → reactive signal** — `startMicMeter()` opens `getUserMedia` audio, builds a Web Audio `AudioContext` analyser (fftSize 512), and each animation frame computes RMS of the time-domain buffer, writing a clamped 0..1 value to `levelRef.current` (scaled ~3.2x). This is wrapped in `try/catch` and is explicitly optional — if it fails the orb still animates from its state. A separate rAF loop scales/brightens the ambient glow ring (`fieldRef`) from the same level.
- 10 **The Decision Core orb (reactive visualization)** — `components/DecisionCore.jsx` renders a `Three.js` icosahedron core + wireframe shell + point light (uses a global `window.THREE`). CFG defines states: idle (calm blue), thinking (fast/bright), resolved (steady green), listening (blue, reactive), speaking (green, rhythmic pulse). When passed a `levelRef`, the animate loop spins up (`react = 1 + lvl*2.2`), scales, and brightens emissive with the live signal. In `CaptainPanel` it appears twice: a small 34px orb in the engine-trace header driven by phase (no `levelRef`, non-reactive), and the 240px orb in the voice overlay driven by `vstate` WITH `levelRef` (mic-reactive).
- 11 **WebGL safety engineering** — The orb intentionally creates its WebGL context ONCE per mount/size; state changes only mutate existing materials in place. A code comment documents why: recreating the renderer on every state flip leaked contexts against the browser's ~16-context cap, which eventually threw and crashed the whole app to a black screen. Context creation and render are wrapped in `try/catch` and it handles `webglcontextlost/restored`, so GPU exhaustion degrades silently instead of crashing.

Key points

- Omnichannel is real as an architecture (thin per-channel adapter over a shared engine) but only the web Captain Panel is a fully working channel end-to-end. WhatsApp parses inbound and produces real replies, but outbound send is a stub echo.
- The WhatsApp phone→captain mapping is a hard-coded 2-entry demo table (`PHONE_TO_CAPTAIN`); production expects a lookup against the captain master.
- Only a POST webhook handler exists. Meta's required GET verify-token handshake for webhook registration is not implemented — a real go-live gap in addition to credentials.

- All voice today is browser-native Web Speech API: STT (SpeechRecognition) for dictation and the conversation loop, TTS (speechSynthesis) for spoken replies. This means Chrome/Edge only, and speech quality/voice availability (especially Indic) depends on the user's browser/OS — the app cannot guarantee a good Indic voice.
- 'Sarvam-ready' is a label and a swap-point comment, not an integration. There is no Sarvam, Whisper, Deepgram, or ElevenLabs code anywhere. PRODUCTION_DELTA lists server-side Whisper ASR + Sarvam TTS as 'wiring pending' / 'Later'.
- Conversation mode is genuinely hands-free and half-duplex (mic muted during TTS to avoid feedback), with tap-to-interrupt — this loop is built and functional in a supported browser.
- Orb reactivity is driven by a REAL mic-amplitude RMS signal (0..1 via levelRef), not a fake animation — but the mapping coefficients (react $1+|v|^{2.2}$, amplitude $*3.2$, scale/emissive multipliers) are hand-tuned magic numbers, consistent with the note that orb reactivity is still being tuned.
- The orb's single-context WebGL design is a deliberate, documented fix for a real crash (context-leak → black screen), showing the reactivity work has already hit and resolved browser-resource limits.
- Chat attachments (Captain Panel) are read into metadata + a downscaled thumbnail, but only filename/mime/size are sent to the backend — actual image/audio understanding (multimodal) is not built (confirmed by PRODUCTION_DELTA).

Status

BUILT AND WORKING: (1) Web Captain Panel chat as a full channel. (2) WhatsApp inbound webhook parsing — both the simplified simulator shape and the real Meta nested shape — wired to the shared engine, producing genuine engine replies keyed per phone number. (3) In-app dictation (single-shot browser STT into the composer). (4) Hands-free Conversation mode: the listen→send→think→speak→listen loop, half-duplex, tap-to-interrupt, 8-language selection, all via browser Web Speech API. (5) The reactive Decision Core orb, including real mic-amplitude reactivity in voice mode and a hardened single-WebGL-context design.

STUB / DEMO-ONLY: WhatsApp OUTBOUND send() — it only echoes the payload (or raises NotImplementedError if creds are set); nothing is delivered to WhatsApp. The phone→captain map is a 2-row demo table. No Meta webhook-verification GET handler exists.

DESIGN-ONLY / NOT BUILT: Server-side speech (Whisper ASR + Sarvam Indic TTS) — 'Sarvam-ready' is a marker only; zero integration code exists. Multimodal understanding of attachments — only metadata is forwarded.

HONEST CAVEATS: Voice is 100% browser-dependent (Chrome/Edge; Indic TTS voice quality/availability varies by device and is not guaranteed). Orb reactivity is functional but its response curve uses hand-tuned constants and is still being tuned. Going live on WhatsApp is blocked on Meta provisioning (business account, verified number, permanent token, public HTTPS webhook), which is not possible in the current sandbox.

Key files

backend/app/channels/whatsapp.py · backend/app/main.py · frontend/src/pages/CaptainPanel.jsx
 · frontend/src/components/DecisionCore.jsx · PRODUCTION_DELTA.md

14. Deployment & Operations

The platform ships as a single Docker image that Render runs as one free-tier web service. A two-stage build compiles the React/Vite frontend and then hands the built files to the Python (FastAPI) backend, which serves both the web UI (at `/`) and the API (at `/api/*`) from the same origin. Deploys are driven by pushing to the GitHub "main" branch: Render reads the `render.yaml` blueprint, rebuilds the image, and restarts the service. All secrets (LLM key, Turso database credentials, the token-signing secret, and the first admin login) are injected as environment variables in the Render dashboard and are never committed to the repo; because the free tier has no persistent disk, all authored data is mirrored to Turso so it survives every redeploy.

How it works

- 1 Stage 1 of the Docker build compiles the frontend** — The Dockerfile starts from `node:20-slim`, runs ``npm ci`` on `frontend/package-lock.json`, copies the frontend source, and runs ``npm run build``, producing static SPA files in `/fe/dist`. (Dockerfile lines 7-13.)
- 2 Stage 2 builds the final backend image and folds in the built UI** — From `python:3.12-slim`, it installs `backend/requirements.txt`, copies the backend source into `/app/backend`, and copies the compiled SPA from stage 1 into `/app/frontend/dist`. The FastAPI app (`main.py`) resolves that dist folder and serves it. (Dockerfile lines 15-32.)
- 3 The container starts one process: uvicorn serving FastAPI** — CMD runs ``uvicorn app.main:app --host 0.0.0.0 --port ${PORT:-8077}``. Render injects `$PORT` at runtime; a plain ``docker run`` defaults to 8077. There is no nginx and no separate frontend server in production — FastAPI serves both API and SPA (same-origin, so Server-Sent Events work without CORS issues). (Dockerfile lines 34-37; `main.py` lines 651-675.)
- 4 FastAPI serves the SPA after all API routes** — `main.py` mounts `/assets` as static files and adds a catch-all GET that returns `index.html` for any non-`/api` path (SPA-friendly routing), while explicitly 404-ing anything under `/api` that has no handler so the API surface is never shadowed. If `frontend/dist` is absent (local dev), mounting is skipped so boot never breaks. (`main.py` lines 656-674.)
- 5 Render provisions the service from the blueprint** — `render.yaml` declares one web service ``valmo-psp``, `env=docker`, `plan=free`, `dockerfilePath ./Dockerfile`, and `healthCheckPath /api/health`. In the dashboard you choose New > Blueprint, connect the GitHub repo, and Render reads `render.yaml` and creates the service. (`render.yaml` lines 7-17; `docs/DEPLOY_RENDER.md` sections 0-1.)
- 6 Operator sets seven secrets in the Render dashboard** — All are declared ``sync: false`` in `render.yaml` so they are never stored in git: `OPENAI_API_KEY`, `OPENAI_BASE_URL` (LLM gateway), `TURSO_DATABASE_URL`, `TURSO_AUTH_TOKEN` (the durable/loss database), `AUTH_SECRET` (signs session tokens), `INITIAL_ADMIN_EMAIL` and `INITIAL_ADMIN_PASSWORD` (the first approver login). `PSP_STATE_DIR=/data` is hardcoded in `render.yaml`; `PORT` is injected automatically by Render. (`render.yaml` lines 18-36; `docs/DEPLOY_RENDER.md` section 2.)
- 7 First boot seeds the initial user and baseline content** — A FastAPI startup hook (`_seed_users`) calls `auth_store.seed_initial()`: if no users exist it creates one approver from `INITIAL_ADMIN_EMAIL/INITIAL_ADMIN_PASSWORD`, or, if those are unset, falls back to a documented default `admin@valmo.local / valmo-admin` and prints a stderr WARNING to change it. It also seeds baseline authored content (KT queue, the Losses domain brain, the COD Shortfall SOP). (`main.py` lines 55-65; `auth/store.py` lines 104-122.)

- 8 **Render polls the health check to decide the service is live** — GET /api/health is public (no auth) and returns {ok: true, active LLM provider name, knowledge corpus stats, and the loss-data source ('remote' Turso / 'local' baked / 'none') with a row count}. Render marks the deploy healthy when this returns 200. (main.py lines 215-225; auth note lines 6-8.)
- 9 **Durable state is mirrored to Turso because the free tier has no disk** — render.yaml sets PSP_STATE_DIR=/data, but the free tier's local disk is wiped on every redeploy and on idle restarts. durable_state.py mirrors each mutable JSON store (users, blueprints, kt_queue, audit rubric/audits, traces, concern_log, cpd) into a Turso key-value table `psp\_state`; the local file is a fast cache and Turso is the durable truth. On boot the app restores authored content from Turso, and seeds re-run. If TURSO_* env vars are unset it degrades to plain local files (local-dev behaviour). (durable_state.py lines 1-16, 33-47; render.yaml lines 2-6.)
- 10 **Redeploys happen on every push to main** — Pushing to the GitHub main branch triggers Render to rebuild the image and redeploy. Because authored state and the team roster live in Turso (not on the ephemeral disk), nothing is lost; the static knowledge corpus is baked into the image and loss data is read from Turso. (docs/DEPLOY_RENDER.md 'Redeploys' section.)
- 11 **Local development uses a separate no-Docker path** — run.sh boots the backend (uvicorn :8077) and frontend (Vite dev server :5190) as two processes, loading credentials via scripts/load_env.sh (env-first, with a baked-file fallback). This is explicitly not the production path — production is the single Docker image above. (run.sh lines 1-24; README 'Run it' section.)

Key points

- Production is ONE Docker image, not a multi-container stack: FastAPI serves both the React SPA and the /api/* endpoints from the same origin and port. There is no nginx in the production image (the nginx flow referenced in _deploy_archive/ is an archived hackathon artifact, explicitly called out as unrelated).
- The build is multi-stage: node:20-slim builds the SPA, then python:3.12-slim serves it plus the API. Only the built /dist is carried into the final image, keeping it small.
- Seven secrets are supplied via the Render dashboard, all sync:false so no real values live in git: OPENAI_API_KEY, OPENAI_BASE_URL, TURSO_DATABASE_URL, TURSO_AUTH_TOKEN, AUTH_SECRET, INITIAL_ADMIN_EMAIL, INITIAL_ADMIN_PASSWORD. .dockerignore additionally keeps backend/data/*.txt (baked key/token files) and backend/data/*.db (loss data) out of the image.
- The health check is GET /api/health (public). Render uses it to confirm the service is up; it also doubles as an operational probe reporting the active LLM provider, knowledge corpus stats, and whether loss data is coming from Turso ('remote'), a baked local DB ('local'), or nothing ('none').
- Durability is via Turso, NOT a Render persistent disk. On the free tier the /data directory is ephemeral; durable_state.py mirrors all mutable JSON stores into a Turso `psp\_state` table so users, authored blueprints/SOPs/KT, the ledger, traces, and audits survive every redeploy and idle restart.
- The free plan sleeps the instance after ~15 minutes of inactivity, so the first request after idle is slow (cold start). This is a documented pilot trade-off, noted directly in render.yaml.
- Safety fallbacks exist for missing secrets but log warnings: without INITIAL_ADMIN_* it seeds admin@valmo.local / valmo-admin, and without AUTH_SECRET it uses an insecure fixed dev string. Both print stderr WARNINGS and are documented as must-change before the pilot.
- Deploys are git-driven: push to main -> Render rebuilds and redeploys. There is no CI/CD pipeline, automated test gate, or GitHub Actions workflow in the repo — the flow is Render's built-in git integration plus manual dashboard steps.

- Session tokens are compact stdlib HMAC-SHA256 tokens (not JWT library) with a 12-hour TTL, signed by AUTH_SECRET (auth/tokens.py).

Status

BUILT AND WORKING: The Dockerfile multi-stage build, the single-service Render blueprint (render.yaml), the public /api/health check, environment-variable secret injection, first-boot seeding of the initial approver and baseline content, HMAC session tokens, and Turso-based durability of authored state (durable_state.py) are all implemented in code, and the docs state the image is verified to build and run. Serving the SPA + API from one FastAPI process is real and exercised. DESIGN/OPERATIONAL CHOICES (not gaps, but worth flagging): the deploy is manual and dashboard-driven — there is no CI pipeline, no automated test gate, and no GitHub Actions in the repo; the free tier is a deliberate pilot choice with real limitations (cold starts after ~15 min idle, ephemeral local disk). DOCUMENTATION DISCREPANCY TO FLAG: docs/DEPLOY_RENDER.md and the state_paths.py docstring describe a Render persistent disk (1 GB mounted at /data) as the durability mechanism, but the current render.yaml uses plan=free with NO disk and relies on Turso mirroring instead (durable_state.py, and render.yaml's own header comment, are explicit that the free tier has no disk). The render.yaml + durable_state.py path is the actual current behaviour; the persistent-disk runbook language is stale. INSECURE FALLBACKS ARE PRESENT BUT GUARDED: if AUTH_SECRET or INITIAL_ADMIN_ are not set the app still boots using documented insecure defaults and logs warnings — safe for local dev, must be overridden for the pilot. No secret values were read or reproduced here.*

Key files

Dockerfile · render.yaml · docs/DEPLOY_RENDER.md · docs/ROLLOUT.md · README.md · run.sh · .dockerignore · backend/app/main.py · backend/app/durable_state.py · backend/app/state_paths.py · backend/app/auth/store.py · backend/app/auth/tokens.py · backend/app/substrate/loss_db.py · backend/requirements.txt · (+1 more)

15. Evaluation & Results

There is no evaluation harness or results dataset in this repo for a COD-shortfall ticket batch, a ~1,045-ticket run, or any quality comparison against a Kapture baseline. Searches across scripts/, backend/scripts/, all Python, docs/, and data files find no such code, no results file, and no notebooks; the string "1,045" / "1045" does not appear anywhere (the single grep hit was a false positive inside a timestamp). A backtest/eval harness of that kind is described only as a plan (docs/ROLLOUT.md §3) and a backlog item (PRODUCTION_DELTA.md #10). What is actually built and working is a batch-audit runner: an LLM-as-judge (audit/runner.py) that scores already-resolved "concerns" against a weighted 6-dimension rubric and persists a 0–100 composite score. As of the checked-in data, exactly one concern has been audited (composite 80), out of 28 concerns in the log.

How it works

- 1 Confirm no COD-shortfall / 1,045-ticket / Kapture-baseline evaluation exists** — Repo-wide search for kapture, 1045/1,045, backtest, replay, ground-truth, scorecard, and per-disposition finds nothing matching an eval of ~1,045 tickets. Kapture is referenced only as the incumbent tool 'being replaced' (README.md line 56) and as an economics figure of Rs 1.2 Cr/quarter (PRODUCTION_DELTA.md line 247) — never as a quality baseline the platform is scored against. No .ipynb notebooks exist anywhere in the repo.
- 2 Note that a proper eval/backtest harness is design-only, not built** — docs/ROLLOUT.md section 3 ('Testing against the last 3 months of tickets — the eval / backtest harness') describes the intended method in future tense: 'A script ingests the ticket dump (CSV export), runs each ticket through the engine, scores the result with the Auditing Studio rubric, and compares the decision to the ticket's recorded outcome' to produce a per-disposition scorecard (correct-resolution %, wrong-way-reversal %, missed-escalation %). No such script exists in scripts/ or backend/scripts/. PRODUCTION_DELTA.md backlog item #10 similarly lists 'Eval harness — golden set + regression gate, LLM-as-judge + human spot-check, online shadow eval, canary + auto-rollback' as to-be-built.
- 3 What is actually built: the batch-audit runner (backend/app/audit/runner.py)** — This is an LLM-as-judge quality auditor, NOT a backtest against human ground truth and NOT a Kapture comparison. `audit_concern(concern_id)` assembles the full picture a QA reviewer would see — the captain's concern, the engine's resolution trace, and the final reply — and asks the deep-tier LLM (the same registry node 'sop_compile' the SOP compiler uses) to score each rubric dimension 0.0–1.0 with a one-line rationale, returned as strict JSON. It retries once on a JSON-parse miss and falls back to zeros on failure.
- 4 Scoring math and persistence** — `_composite()` computes a weighted score 0–100, normalized by the sum of dimension weights, clamping each dimension score to [0,1]. Each result (per-dimension scores + rationales, composite, rubric_version, timestamp) is appended to `data/audits.json` under a lock.
- 5 Batch sampling behavior** — `audit_batch(limit=10)` audits the most recent N UN-audited concerns only (newest-first from the concern log, skipping any already in `audits.json`) — sampling by design, not a full sweep. It returns `{audited, avg_composite, results, already_audited}`. There is no built-in notion of a fixed 1,045-ticket set; N is a caller-supplied limit defaulting to 10.
- 6 The rubric being scored against (backend/data/audit_rubric.json, version 1)** — Six weighted dimensions summing to 1.0: identification 0.20, grounding 0.15, decision 0.20, partner_supportedness 0.25, efficiency 0.10, safety 0.10. The rubric is editable/versioned via `rubric.py` (`save_rubric` bumps the version); each audit is stamped with the `rubric_version` so scores stay comparable within a version.
- 7 Aggregation for the dashboard** — `scores()` reads `audits.json` and returns count, overall avg composite, avg per dimension, a by-day time series, and a breakdown by disposition. Exposed via

FastAPI (backend/app/main.py): POST /api/audit/run, POST /api/audit/run_batch, GET /api/audit/scores, GET/POST /api/audit/rubric. The frontend 'Auditing Studio' (AuditScores in SupportCommand.jsx) renders the rubric editor and this scores dashboard.

- 8 **Actual results present in the repo** — data/audits.json contains exactly ONE audit record: concern CNC-EC1CAD1B, disposition hardstop_loss, action escalate, composite = 80 (identification 0.85, grounding 0.75, decision 0.80, partner_supportedness 0.70, efficiency 0.95, safety 0.90), rubric_version 1, audited 2026-07-10. data/concern_log.json holds 28 concerns total, so 27 remain un-audited. This is clearly local demo/dev data, not a systematic evaluation.
- 9 **Related but separate: CPD / satisfaction loop (backend/app/audit/cpd.py)** — record_satisfaction() logs whether a captain was satisfied after a resolution; dissatisfaction becomes a Continuous-Problem-Discovery item. satisfaction_stats() computes a CSAT %. This is a feedback-capture loop, not an accuracy evaluation, and data/cpd_log.json is essentially empty (233 bytes).

Key points

- No COD-shortfall ticket batch, no ~1,045-ticket run, and no Kapture quality-baseline comparison exist anywhere in the repo — not as code, data, notebooks, or results. The one '1045' grep match was a false positive inside a timestamp string in concern_log.json.
- The eval/backtest harness that would replay historical tickets against known human outcomes to produce a per-disposition scorecard is DESIGN-ONLY: described prospectively in docs/ROLLOUT.md §3 and listed as backlog item #10 in PRODUCTION_DELTA.md. No implementing script is present.
- The only evaluation mechanism that is actually built is an LLM-as-judge quality auditor (audit/runner.py). It scores the engine's OWN resolutions against an internal rubric — it does not compare against Kapture, and it does not verify decisions against human ground truth.
- audit_batch samples the most recent N un-audited concerns (default 10); it is not a fixed large-batch benchmark.
- Rubric v1 has 6 weighted dimensions (partner_supportedness weighted highest at 0.25); composite is a weight-normalized 0–100.
- The only concrete result checked into the repo is a single audited concern with composite score 80; 27 of 28 logged concerns are un-audited.
- Any single-number 'quality' figure a reader might expect (accuracy %, solve rate vs Kapture) is NOT computed or stored anywhere in this codebase.

Status

BUILT AND WORKING: the batch-audit runner (audit/runner.py) — LLM-as-judge scoring of resolved concerns against an editable, versioned 6-dimension rubric, with weighted 0–100 composite, persistence to audits.json, batch sampling of un-audited concerns, and a scores() aggregation surfaced through /api/audit/ endpoints and the frontend Auditing Studio. DATA PRESENT IS DEMO-SCALE ONLY: exactly one audit record (composite 80) against 28 logged concerns; not a systematic evaluation and not a statistically meaningful result set. NOT BUILT (design/backlog only): the eval/backtest harness that replays historical tickets and compares engine decisions to known human outcomes for a per-disposition accuracy scorecard (ROLLOUT.md §3; PRODUCTION_DELTA #10). DOES NOT EXIST AT ALL: any COD-shortfall ticket batch, any ~1,045-ticket evaluation run, and any quality-metrics comparison against a Kapture baseline — no such code, data, or results are in the repo. Kapture appears only as the tool being replaced (README) and as a cost figure (PRODUCTION_DELTA), never as a measured quality benchmark. I did not fabricate any accuracy/solve-rate numbers because none exist in the codebase.*

Key files

backend/app/audit/runner.py · backend/app/audit/rubric.py · backend/app/audit/cpd.py ·
backend/data/audit_rubric.json · backend/data/audits.json · backend/data/concern_log.json ·
backend/data/cpd_log.json · backend/app/main.py · docs/ROLLOUT.md · PRODUCTION_DELTA.md ·
frontend/src/pages/SupportCommand.jsx

Appendix A — Recent Hardening & Feature Cycle

The following work was delivered in the most recent development cycle. Items that touched core resolution, money, or data paths were adversarially reviewed (independent agents attempting to break each change) before shipping; defects found in review were fixed prior to release.

Durable state on the free tier (Turso)

Team-authored knowledge (SOPs, Brains, the governance framework, user accounts) is persisted to Turso (libSQL over HTTP) with a local-file fallback, so content survives the Render free tier's ephemeral restarts. Reads are clobber-safe: a transport error returns 'not confirmed' so a store never overwrites durable authored data with baked-in seeds on a transient blip.

Three-level knowledge model (Domain → Disposition → SOP)

Separated an SOP's scenario name (title) from the concern category (disposition), which had been conflated. The engine now routes on disposition and the taxonomy is emergent — the system proposes new category keys rather than requiring them to be hand-defined. SOPs from the source sheet and the collated SOP document were seeded and enriched (not duplicated).

Authoring fidelity

The SOP compiler preserves operational specifics verbatim — form links, point-of-contact escalation chains, and the exact captain-facing reply — and a fidelity check flags anything the compilation dropped BEFORE the author approves, turning 'hope the machine understood it' into 'it shows what it missed, you fix it.'

Core-workflow audit — 32 findings, high/medium fixed

A structured audit (parallel reviewers per subsystem, adversarially verified) surfaced 32 confirmed issues. All 4 high-severity and 8 medium-severity were fixed and shipped: a cross-site-scripting escape gap, a data-loss race on a durable-store blip, a reply that could wrongly imply money was reversed on an escalation, an auth 500 on malformed tokens, disposition routing that never fired, unlocked concurrent writes in three stores, and more. A dead trace feature was removed.

Governance conformance loop

New: a compiled SOP is scored against the organisation's approved Governance Framework's mandates — an accountable owner, a rupee cap on any money-moving action, a cited partner right, idempotency on money moves, and a priority band. It appears as a step in the compile animation, is stamped on every stored SOP, is available on demand, and the Approve action gates on high-severity violations (with an advisory override). Adversarial review caught and closed a bypass where an action beginning with an informational verb could hide a later money move.

Audit cleanup

Four lower-severity bugs fixed (a Brain edit that could orphan/duplicate on a domain change; a gap rendered twice; a multi-concern turn whose trace saved under only one concern; resolved cases double-counted in stats) and a batch of dead code removed.

One reactive motion language

The 'Decision Core' orb was generalised into a reusable reactive component driven by a live signal: mic amplitude in voice mode, and the live event stream on the monitoring view (the always-on orb now pulses as events are consumed). Callers without a signal are unchanged. Voice-mode reactivity is still being tuned.

Voice / conversation mode

In-app voice conversation using the browser Web Speech API (speech-to-text and text-to-speech), a hands-free listen→answer→listen loop, and a full-screen visualiser. Designed to swap to a production Indian-language voice provider (Sarvam) without pipeline changes.

Team access & leadership onboarding

The approver-only team panel was made first-class for onboarding leadership: plain-language roles (Admin / Editor / Viewer) with a live description of what each grants, a one-click strong temp-password generator, and a credential-handoff block that copies a ready-to-send invite. Verified end-to-end (an admin creates an admin account and it logs in).